AUTOMATED WATER FILTRATION DESIGN AND RECOMMENDATION SYSTEM

A MAJOR PROJECT REPORT

submitted by

Alakananda.J.Nair

KH.SC.I5MCA23021

Under the supervision of

Rajitha M.R

Assistant Professor

Department of Computer Science and IT

in partial fulfilment of the requirement of

AMRITA VISHWA VIDYAPEETHAM

for the award of the degree of

INTEGRATED MCA

April 2026

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956
KOCHI CAMPUS

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **AUTOMATED WATER FILTRATION DESIGN AND RECOMMENDATION SYSTEM** submitted by **Alakananda.J.Nair – KH.SC.I5MCA23021** in partial fulfilment of the requirements for the award of the **DEGREE OF INTEGRATED MCA** is a bonafide record of the work carried out under my guidance and supervision at School of Computing, Amrita Vishwa Vidyapeetham, Kochi Campus.

Rajitha M R
Project Advisor
Assistant Professor
Department of Computer
Science & IT
School of Computing
Amrita Vishwa Vidyapeetham
Kochi Campus

Dr Sangeetha J
Head, Department of Computer
Science & IT
School of Computing
Amrita Vishwa Vidyapeetham
Kochi Campus

The project was evaluated as on: _____

Internal Examiner **External Examiner**

DECLARATION

I affirm that the project work entitled "**AUTOMATED WATER FILTRATION DESIGN AND RECOMMENDATION SYSTEM**" being submitted in partial fulfilment for the award of the **DEGREE OF INTEGRATED MCA** is the original work carried out by me. It has not formed the part of any other project work submitted for the award of any degree or diploma, either in this or any other University.

Place: Kochi Alakananda.J.Nair – KH.SC.I5MCA23021
Date: _____

Organization Certificate

Document No - CIALHR0013v1.1

CHEMSBURY

30th Mar 2026

TO WHOM IT MAY CONCERN

This is to formally certify that **Ms. Alakananda J Nair (KH.SC.I5MCA23021)**, a student of Integrated MCA (2023 Batch), Third Year, of Amrita Vishwa Vidyapeetham, has successfully completed the Major Project titled: **Automated Water Filtration Design and Recommendation System**

The above project was undertaken at Chemsbury Environmental Solutions during the period from January 2026 to April 2026, under the guidance and supervision of the undersigned.

The scope of the project included the analysis of raw water quality laboratory parameters and the development of a structured methodology for recommending suitable water filtration and treatment systems. The work involved system selection, process validation, and optimization in accordance with established industry practices and applicable standards.

Her has successfully completed all assigned objectives, including the design, development, and implementation of a software-based solution addressing practical challenges in water treatment system selection. Her performance throughout the project tenure was found to be diligent, technically competent, and professional.

This certificate is issued upon the successful completion of the project for academic purposes only and does not entail any commercial obligation on the part of the organization.

We appreciate her sincere efforts and wish her continued success in all future academic and professional endeavors. We appreciate their sincere efforts and wish them success in their future endeavors.

Mentor / Authorized Signatory

J B Nair
General Manager
Chemsbury Environmental Solutions

Contact: 70346 66768
Email: gm@chemsbury.in

CHEMSBURY ENVIRONMENTAL SOLUTIONS
Hill Palace Road
Tripunithura - 682301

CHEMSBURY ENVIRONMENTAL SOLUTIONS
TMC NO. 21/586 | ATHANICKAL BUILDING | EASTFORT GATE | HILL PLACE ROAD | TRIPUNITHURA – 682031
PHONE: 91 4844619213 / Email:info@chemsbury.com Web: www.chemsbury.in

DEDICATION

To

My parents, all my teachers and the eternal God.
Thank you for believing in me and encouraging me throughout.

ACKNOWLEDGEMENT

A venture can't be completely by themselves. I take this opportunity to gratefully acknowledge various people who acted as guides along the way.

The success of any work require the blessings of the Lord Almighty. I thank my God for aiding me in my travel to success.

I offer my humble salutations at the lotus feet of Sri Mata Amritanandhamayi Devi, who is guiding light of this research work.

I express my thankfulness to **Dr. U. Krishnakumar**, Director, Amrita Vishwa Vidyapeetham, Kochi Campus for giving me an opportunity to do my project.

I would like to express my deep gratitude to **Dr Sangeetha J**, Head of the Department, Mathematics, School of Physical Sciences, Amrita Vishwa Vidyapeetham, Kochi Campus for her valuable guidance and for the support she rendered on me.

I would also like to thank my guide **Rajitha M R**, Asst. Professor who revised my ideas and helped me to bring it to the research paper.

I would also like to express my gratitude to the staff of the ICTS Department, School of Physical Sciences, Amrita Vishwa Vidyapeetham, Kochi Campus for the technical support they gave.

This thanks giving cannot be complete without mentioning my friends and parents who gave the mental strength which I almost lost in between the journey.

Alakananda.J.Nair – KH.SC.I5MCA23021

TABLE OF CONTENT PageNo

INTRODUCTION

1.0 INTRODUCTION

1.1 About the System

The system is designed to automate the process of water filtration system design by analyzing water quality parameters and generating appropriate filtration solutions. Water quality is determined by various physical and chemical parameters such as Total Dissolved Solids (TDS), hardness, turbidity, iron content, chloride levels, pH, and the presence of gases like hydrogen sulfide. These parameters directly influence the selection of filtration methods and media.

Total Dissolved Solids (TDS) represent the amount of dissolved substances in water. High TDS levels can affect taste and require advanced filtration methods such as Reverse Osmosis. Hardness in water is caused by calcium and magnesium ions and leads to scaling in pipes and equipment. Turbidity indicates the presence of suspended particles, which can be removed using sand filtration techniques.

Iron and hydrogen sulfide are common contaminants that affect water quality by causing unpleasant taste, odor, and staining. These are typically removed using oxidation processes and activated carbon filtration. Chloride levels influence the corrosiveness and taste of water, while pH determines whether the water is acidic or alkaline.

To process these parameters efficiently, the system uses Optical Character Recognition (OCR) technology, which extracts data from water test reports in image format. OCR converts printed or handwritten text into digital data, reducing manual effort and improving accuracy. The system then applies predefined rules and engineering logic to analyze the input values.

Based on this analysis, the system automatically selects suitable filtration techniques such as activated carbon filtration, reverse osmosis, sand filtration, and ion exchange methods. It calculates the required media quantity, determines vessel size, and generates a complete filtration design along with a bill of materials and visual representation.

By integrating automation, data processing, and engineering principles, the system provides a fast, accurate, and standardized solution for designing water filtration systems. It reduces dependency on manual work, minimizes errors, and improves efficiency, making it highly useful for engineers, technicians, and users.

2.0 NEED FOR THE SYSTEM

The need for an automated water filtration design system arises due to the limitations and challenges associated with the existing manual approach. In the current system, engineers and technicians are required to manually analyze water test reports, interpret multiple water quality parameters, perform calculations, and design filtration systems. This process is not only time-consuming but also requires a high level of expertise.

One of the major reasons for developing this system is to reduce the time required for designing filtration systems. Manual analysis involves repetitive tasks such as reading reports, comparing values with standard limits, and performing calculations for each case. When multiple reports need to be processed, the workload increases significantly, making the process inefficient.

There is also a growing need for faster and more efficient solutions. In real-world applications, organizations require quick turnaround times and accurate results. Manual methods are not capable of meeting these demands effectively.

Another important need is to reduce human errors. Since the system relies on manual data interpretation and calculations, mistakes can occur in reading values, applying formulas, or selecting filtration media. These errors can lead to incorrect system design, which may affect water quality and system performance.

The existing system also lacks standardization. Different engineers may use different approaches or methods, resulting in inconsistent outputs for the same input data. This reduces the reliability of the system and creates confusion for users and customers.

Additionally, customers and users expect transparency and clarity in system design. They want to understand how filtration decisions are made and what materials are used. The manual system does not always provide clear and structured outputs, making it difficult for users to interpret the results.

The proposed system addresses all these needs by providing an automated solution that simplifies the design process. It reduces manual effort, minimizes errors, ensures consistency, and generates clear and structured outputs. The system improves efficiency, saves time, and enhances the overall reliability of water filtration system design.

3.0 BACKGROUND STUDY

3.1 Existing System

The existing system for designing water filtration solutions is primarily manual and depends heavily on human expertise. Engineers and technicians analyze water test reports provided by laboratories, which contain various parameters such as Total Dissolved Solids (TDS), hardness, turbidity, iron, chloride, pH, and hydrogen sulfide. Based on these values, they manually decide the appropriate filtration methods and system configuration.

In the current approach, the interpretation of water quality parameters is done by experienced professionals who rely on their knowledge and past experience. They compare the values with standard limits and select suitable filtration media such as activated carbon, sand filters, ion exchange resins, or reverse osmosis systems. This process requires a deep understanding of water treatment principles and can vary from person to person.

Calculations involved in the design process, such as determining media quantity, flow rate requirements, and vessel size, are typically performed using manual methods or spreadsheet tools. Engineers must apply formulas, consider contact time, and ensure proper layering of filtration media. These calculations are repetitive and require careful attention to avoid errors.

Additionally, the preparation of system design outputs, including bill of materials (BOM), vessel configuration, and graphical representation, is carried out manually. Engineers often create diagrams and reports using separate tools, which increases the time required to complete the design process.
The existing system also lacks integration between different stages of the process. Data input, analysis, calculation, and output generation are handled separately, leading to inefficiencies and increased chances of errors. There is no centralized system to manage and automate these tasks.

Overall, the existing system is functional but inefficient, as it is time-consuming, error-prone, and dependent on individual expertise. This highlights the need for an automated system that can simplify the process and provide consistent and accurate results.

3.2 Drawbacks

The existing manual system for designing water filtration solutions has several

limitations that affect efficiency, accuracy, and reliability. One of the major drawbacks is that the process is highly time-consuming. Engineers need to analyze water test reports, perform calculations, and prepare system designs manually, which takes a significant amount of time, especially when handling multiple reports.

Another important drawback is the high chance of human errors. Since the system relies on manual data interpretation and calculations, mistakes can occur in reading values, applying formulas, or selecting appropriate filtration media. These errors can lead to incorrect system designs, which may affect water quality and system performance.

The existing system also depends heavily on individual expertise. Only experienced engineers can accurately interpret water parameters and design suitable filtration systems. This makes it difficult for beginners or less experienced users to perform the task effectively, limiting accessibility.

Lack of standardization is another major issue. Different engineers may follow different methods or approaches, resulting in inconsistent outputs for the same input data. This reduces the reliability of the system and creates confusion among users and customers.

Manual calculations and repetitive tasks further increase the workload for engineers. They need to perform similar operations repeatedly for each project, which can lead to fatigue and reduced productivity. Additionally, preparing reports, diagrams, and bill of materials manually slows down the overall process.

The system also struggles to handle large volumes of data efficiently. Managing multiple water reports and designing systems for each one becomes challenging without automation. Furthermore, there is limited transparency, as customers may not clearly understand how the final design decisions are made.

3.3 Proposed System

The proposed system is an **Automated Water Filtration Design and Recommendation System** developed to overcome the limitations of the existing manual process. The system is designed to provide a fast, accurate, and standardized solution for designing water filtration systems based on water quality parameters.

In this system, users can input water quality data through two methods: manual data entry and Optical Character Recognition (OCR). In manual entry, users can directly enter values such as TDS, hardness, turbidity, iron, chloride, pH, and hydrogen sulfide using structured forms. In OCR-based input, users can upload images of water test reports, and the system automatically extracts the required values using OCR technology. This reduces manual effort and improves efficiency.

Once the data is collected, the system processes the input using predefined rules and engineering logic. Each parameter is analyzed based on standard threshold values. For example, if the iron

content exceeds a certain limit, the system selects appropriate filtration media such as manganese dioxide or Birm. Similarly, high TDS levels may result in the recommendation of reverse osmosis systems, while high turbidity may lead to the selection of sand filtration methods.

The system then performs necessary calculations to determine the required media quantity and vessel size. It uses standard formulas and considers factors such as flow rate and contact time

to ensure accurate design. The system also determines the correct layering of filtration media to ensure proper functioning.

After processing, the system generates output in multiple forms. It provides a **Bill of Materials (BOM)** listing all required materials and their quantities. It also generates a visual representation of the filtration system, showing the arrangement of different media layers. Additionally, a detailed report is created, which can be exported as a PDF for easy sharing and documentation.

The proposed system integrates all stages of the process, including data input, processing, calculation, and output generation, into a single platform. This improves efficiency and reduces dependency on manual work. It ensures consistency by applying the same rules for all inputs, resulting in standardized outputs.

Overall, the proposed system provides a reliable, efficient, and user-friendly solution for water filtration system design. It reduces human errors, saves time, improves accuracy, and enhances productivity, making it highly beneficial for engineers, technicians, and users.

4.0 PROBLEM FORMULATION

4.1 Main Objectives

The main objective of this project is to develop an automated system that converts raw water quality data into an accurate and standardized water filtration system design. The system aims to simplify the complex process of analyzing water test reports and selecting appropriate filtration methods.

The project focuses on reducing manual effort by automating key tasks such as data input, parameter analysis, media selection, and vessel calculation. By doing so, it minimizes the chances of human errors and ensures more reliable results.
Another important objective is to ensure consistency in the design process. The system follows predefined rules and engineering standards, so that the same input data always produces the same output. This improves the reliability and trustworthiness of the system.

The system also aims to improve efficiency by reducing the time required to design filtration systems. It enables faster processing of multiple reports and helps engineers and technicians complete their work more quickly.
Additionally, the project aims to provide clear and user-friendly outputs such as bill of materials, system configurations, and visual representations. This helps users and customers easily understand the design.

Overall, the objective of the system is to provide a fast, accurate, efficient, and standardized solution for water filtration system design, reducing dependency on manual work and improving productivity.

4.2 Specific Objectives

The specific objectives of this project are designed to achieve automation, accuracy, and efficiency in water filtration system design. One of the primary objectives is to automate the interpretation of water quality reports using both manual input and Optical Character Recognition (OCR). This reduces the need for manual data entry and speeds up the overall process.

Another objective is to apply standardized rules and engineering logic for selecting appropriate filtration media based on different water quality parameters. This ensures that the system provides consistent and reliable results for the same input data.

The system also aims to accurately calculate important design parameters such as media quantity, flow rate requirements, and vessel size. These calculations are performed automatically using predefined formulas, reducing the risk of human errors.

Additionally, the project focuses on generating a detailed Bill of Materials (BOM), which includes the list of required filtration media and their quantities. This helps in easy planning and implementation of the filtration system.

Providing a clear visual representation of the filtration system is another key objective. The system generates diagrams showing the arrangement and layering of filtration media, making it easier for users to understand the design.

Finally, the system aims to improve efficiency, reduce workload, and simplify the overall design process, making it accessible even for users with less technical expertise.

4.3 Methodology

The development of the proposed system follows a systematic and structured approach to ensure accuracy, efficiency, and reliability. The methodology is divided into several phases, each focusing on a specific stage of the system development process.

The first phase is **Requirement Analysis**, where the system requirements are identified and analyzed. In this phase, water quality parameters, filtration techniques, and existing manual processes are studied in detail. The limitations of the current system are identified, and the requirements for the new system are clearly defined.

The second phase is **System Design**, where the overall structure of the system is planned. This includes designing the system architecture, database structure, module organization, and user interface. Flow diagrams and design models are created to represent how the system will function.

The third phase is **Implementation**, where the actual development of the system takes place. Different modules such as OCR input, manual data entry, filtration logic, calculation engine, and report generation are developed using appropriate technologies. Each module is built step by step and tested individually.

The next phase is **Integration**, where all the developed modules are combined into a single system. The interaction between different modules is tested to ensure smooth data flow and proper functionality.

The fifth phase is **Testing and Validation**, where the system is tested to ensure that it works correctly. This includes checking the accuracy of calculations, performance of OCR, correctness of outputs, and overall usability of the system. Errors are identified and fixed during this phase.

Finally, the system enters the **Deployment** phase, where it is made available for practical use. The system is installed, and users can start using it for designing water filtration systems. Feedback is collected to further improve the system.
This structured methodology ensures that the system is developed in an organized manner, resulting in a reliable, efficient, and user-friendly application.

4.4 Platform Selection

- **Frontend:** Android Studio for mobile application and basic web technologies (HTML, CSS) for web interface
- **Back-End (Server-Side):** Java (Android) for implementing system logic and processing
- **Database**: **SQLite** for storing user data and system information locally
- **OCR Tool: Google ML Kit (OCR)**for extracting data from water test reports and convert it into digital data**.**
- **Graphics**: Android Canvas / XML layouts for visual representation of filter design
- **Report Generation:** PDF libraries in Android for generating reports

SYSTEM ANALYSIS

5.1 System Analysis

System analysis is a crucial phase in the development of any software system, as it involves a detailed study of the existing system, identification of its limitations, and definition of requirements for the proposed system. The main objective of system analysis is to understand how the current process works and determine how it can be improved using technology.

In the context of water filtration system design, the existing process is largely manual and depends on human expertise. Engineers are required to analyze water test reports, which contain multiple parameters such as Total Dissolved Solids (TDS), hardness, turbidity, iron, chloride, pH, and hydrogen sulfide. Each of these parameters must be carefully interpreted and compared with standard limits to determine the appropriate filtration method.

The manual nature of this process introduces several challenges. It is time-consuming, especially when handling multiple reports. Engineers must perform repetitive calculations for each project, which increases workload and reduces efficiency. Additionally, the process is prone to human errors, which can lead to incorrect system design and affect water quality.

Another important aspect identified during system analysis is the lack of integration between different stages of the process. Data entry, parameter analysis, calculation, and report generation are handled separately, often using different tools. This lack of a unified system leads to inefficiencies and increases the chances of errors.

The system analysis also highlights the need for standardization. In the current system, different engineers may follow different approaches, resulting in inconsistent outputs for the same input data. This reduces the reliability of the system and creates confusion among users and customers.

To address these issues, the proposed system introduces automation and integration. It combines all stages of the process into a single platform, allowing users to input data, process it, and generate outputs within the same system. The use of OCR technology further enhances the system by automating data extraction from water test reports.

The system also incorporates predefined rules and formulas to ensure consistent and accurate results. By standardizing the design process, the system ensures that the same input data always produces the same output, improving reliability.

In addition, the proposed system improves usability by providing clear outputs such as bill of materials, system configuration, and visual representations. These outputs are designed to be easily understood by users, reducing the need for expert interpretation.

Overall, the system analysis clearly demonstrates that the proposed system addresses the limitations of the existing system by improving efficiency, accuracy, consistency, and usability. It provides a well-integrated and automated solution that enhances the overall process of water filtration system design.

5.2 Feasibility Analysis

Feasibility analysis is conducted to evaluate whether the proposed system is practical, viable, and beneficial to implement. It helps in understanding the technical, economic, and performance aspects of the system before development and deployment. The feasibility study ensures that the system can be successfully implemented without major constraints and will provide the expected benefits.

The proposed system is analyzed based on three main aspects: technical feasibility, economical feasibility, and performance feasibility.

5.2.1 Technical Analysis

Technical analysis evaluates whether the required technology, tools, and resources are available to develop and implement the proposed system. The system is developed using Android Studio, Java, SQLite, and Google ML Kit for OCR, which are widely used and well-supported technologies.

Android Studio provides a robust platform for building mobile applications with user-friendly interfaces. Java is used for implementing the system logic, handling data processing, and performing calculations. SQLite is a lightweight database that efficiently stores and manages data locally within the application.

Google ML Kit (OCR) is used for text recognition, which enables the system to extract water quality parameters from water test report images and convert them into digital format. This reduces manual data entry, improves accuracy, and increases processing speed.

The system does not require high-end hardware and can run on standard Android devices. The integration of different modules such as OCR, data processing, and report generation is technically feasible. Therefore, the proposed system is technically reliable, easy to implement, and maintainable.

5.2.2 Economical Analysis

Economical analysis determines whether the proposed system is cost-effective and financially feasible. The system is designed using cost-efficient and widely available tools such as Android Studio, Java, SQLite, and Google ML Kit, which reduces development and operational costs.

The system eliminates the need for expensive infrastructure, as it can run on standard devices without requiring dedicated servers. It reduces manual work, saving time and effort for engineers and technicians. This leads to increased productivity and reduced labor costs.
By minimizing errors in calculations and design, the system also helps avoid additional costs that may arise from incorrect system implementation. The automated report generation further reduces the need for manual documentation, saving time and resources.

5.2.3 Performance Analysis

Performance analysis evaluates how efficiently the system performs under different conditions. The proposed system is designed to provide fast and accurate results by automating data processing and calculations.

The system quickly processes input data, whether entered manually or extracted using OCR, and generates filtration system designs within a short time. It reduces delays associated with manual calculations and improves overall efficiency.

The system is capable of handling multiple inputs and ensures consistent performance without significant delays. It provides accurate outputs with minimal errors, improving reliability.
Additionally, the system ensures a smooth user experience through well-designed interfaces and efficient data handling. The use of optimized algorithms and lightweight database management enhances system performance.

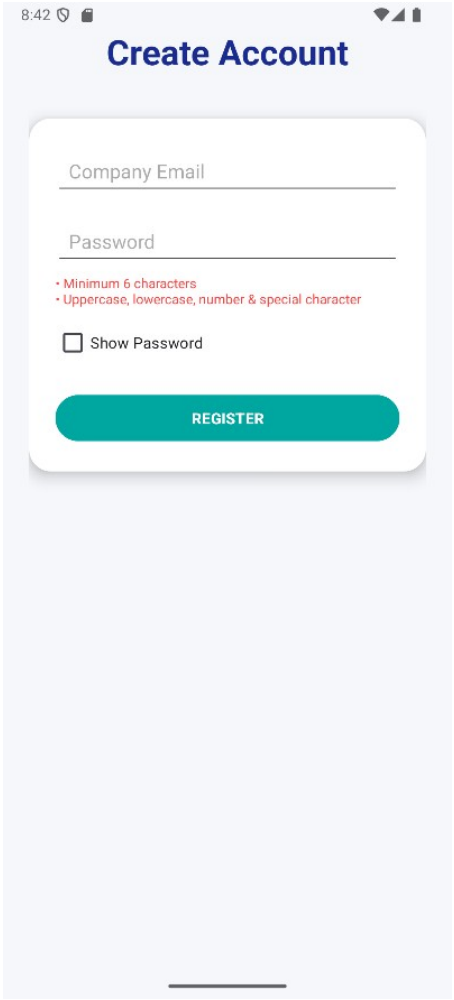
Overall, the proposed system demonstrates high performance, efficiency, and reliability in handling water filtration system design tasks.

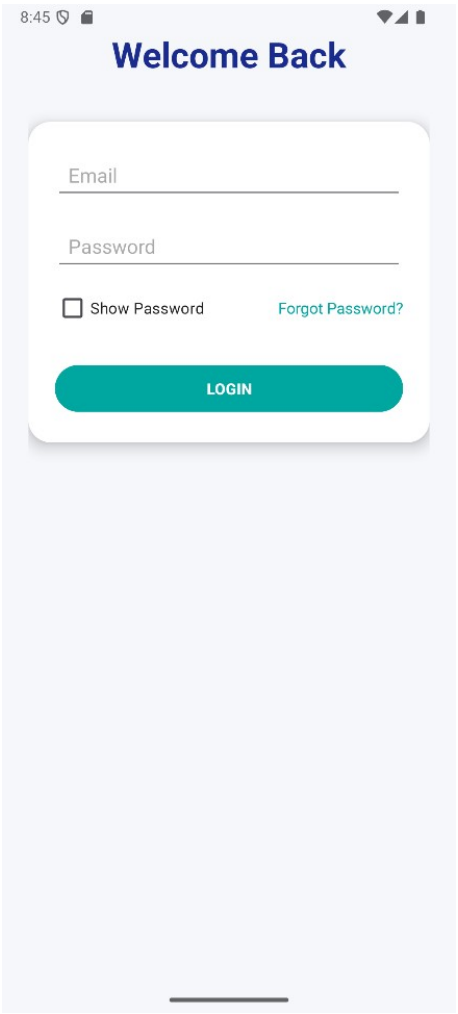
SYSTEM DESIGN

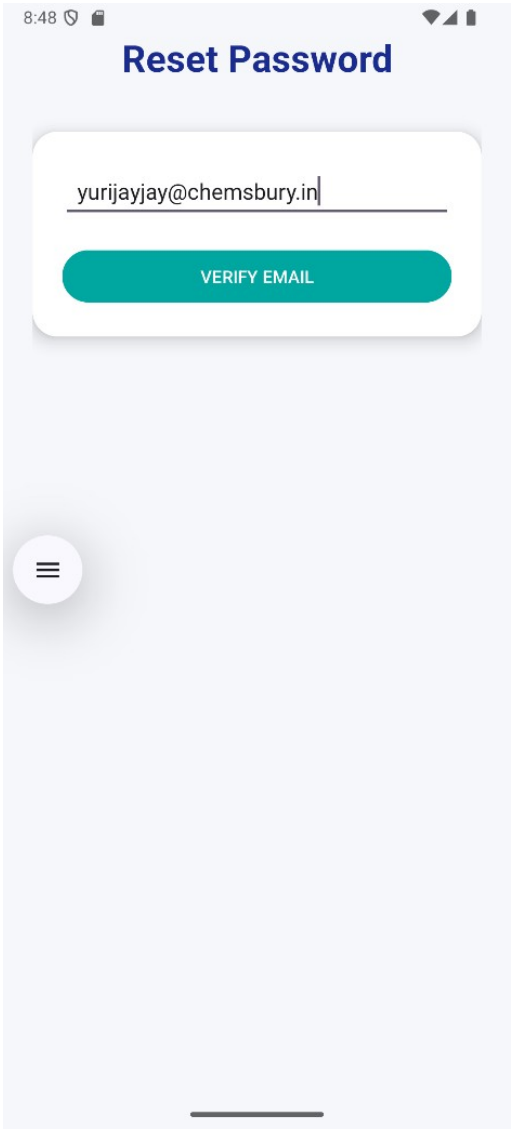
6.1 Input Design

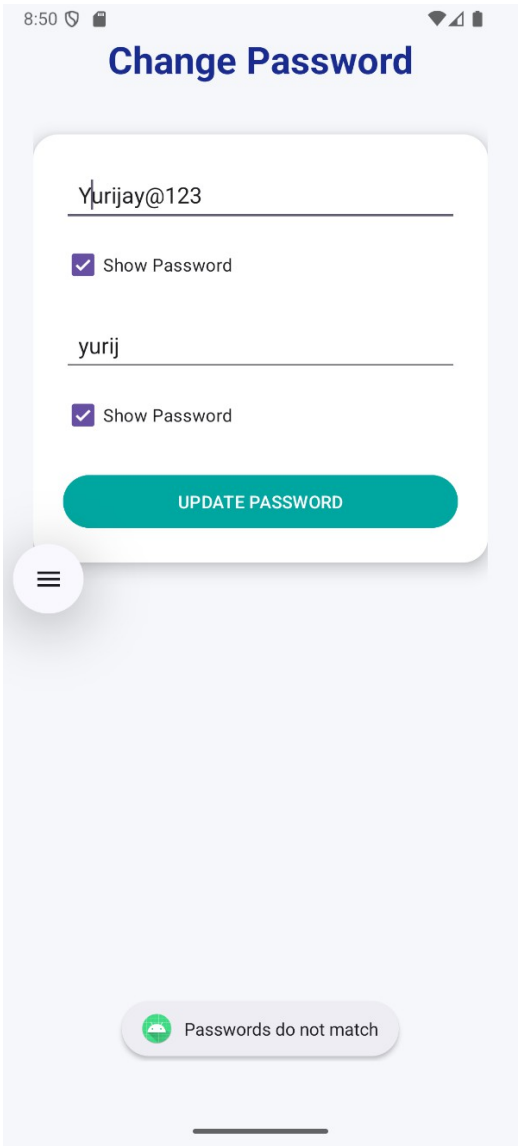
Input design focuses on creating simple, user-friendly, and accurate data entry interfaces. The system provides clean and structured forms to ensure easy input of data.

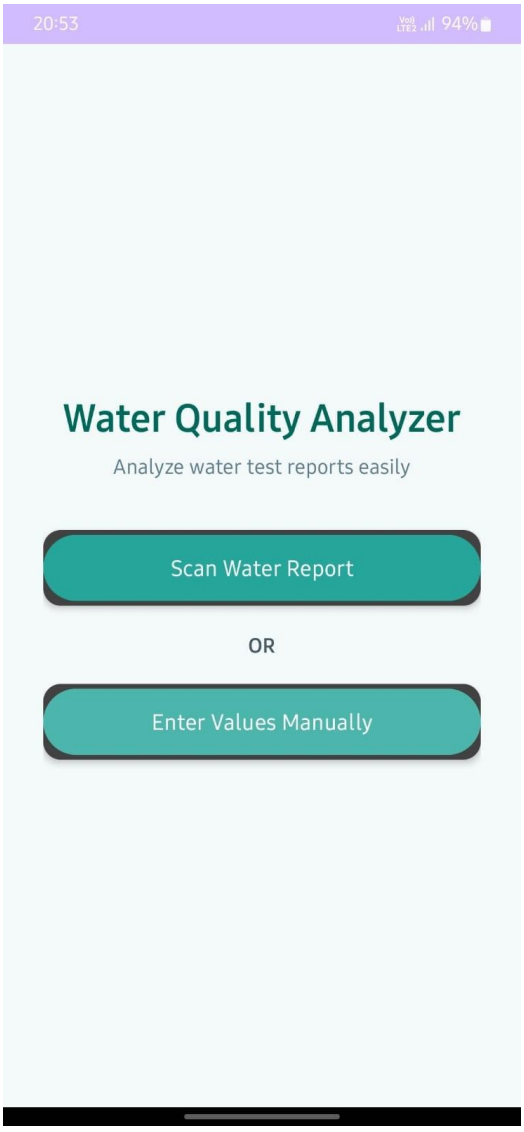
- **Registration Forms:** Separate forms for user registration with fields like email and password, including validation to avoid incorrect data.
- **Login Form**: A secure login page that verifies user credentials and provides access to the system.
- **Data Entry Forms**: Structured forms for entering water quality parameters such as TDS, hardness, turbidity, iron, and chloride with proper validation.
- **OCR Input**: Allows users to upload water test report images, where the system extracts values automatically using OCR.
- **Validation Features**: Ensures correct and complete data entry by checking input values and allowing users to edit extracted data**.**

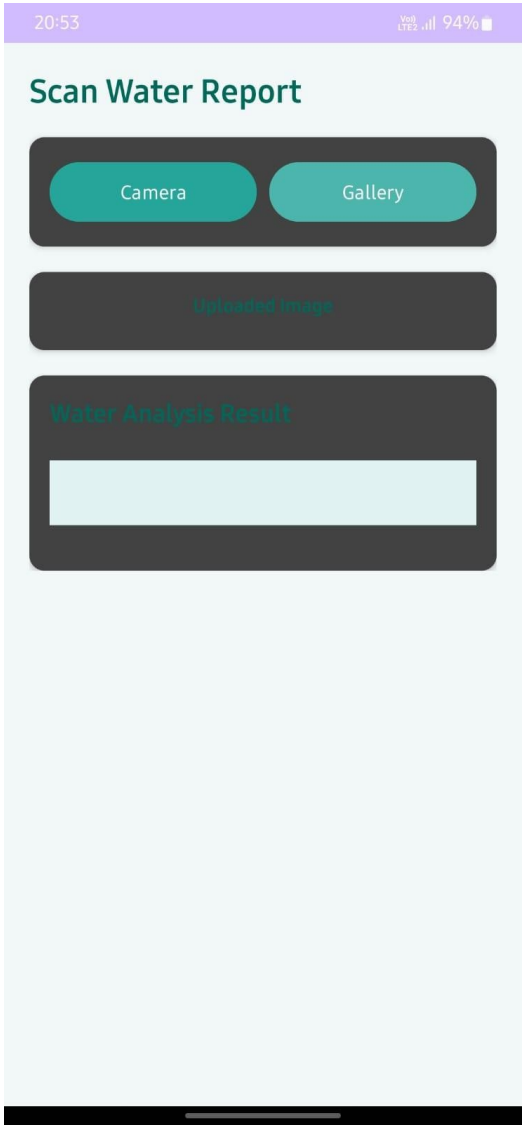




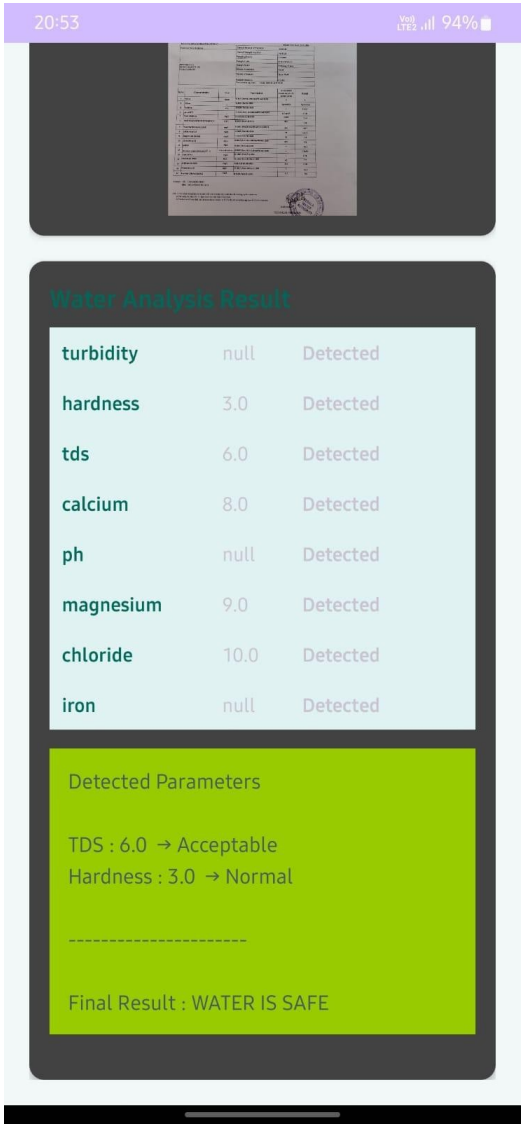

8:48
Reset Password
yurijayjay@chemsbury.in
VERIFY EMAIL

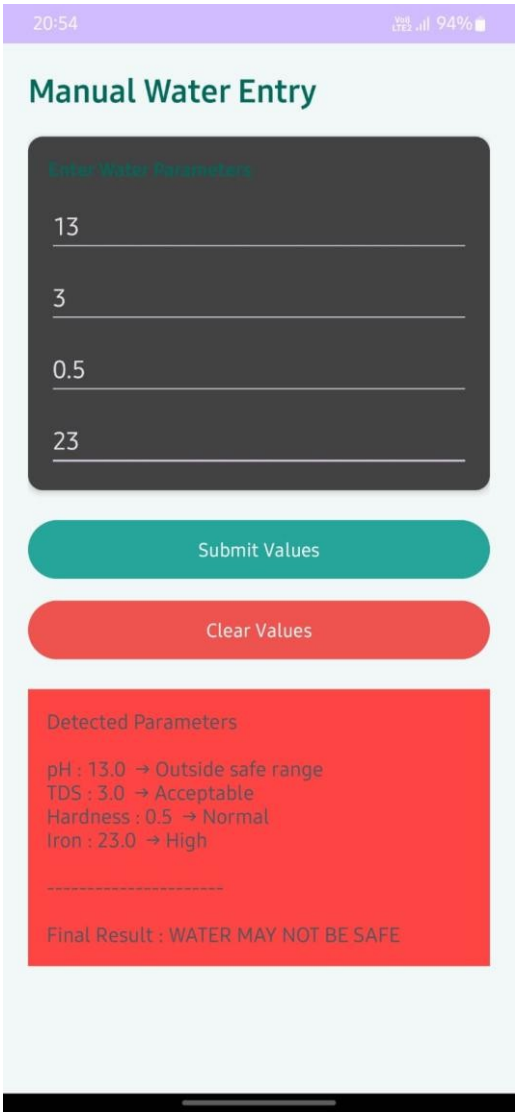

8:50
Change Password
Yurijay@123
Show Password
yurij
Show Password
UPDATE PASSWORD
Passwords do not match

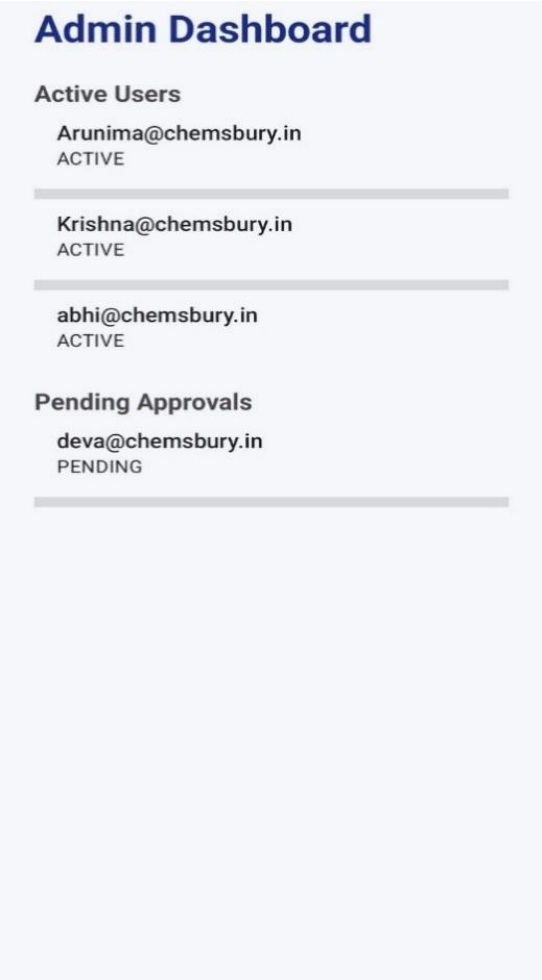

20:53
94%
Water Quality Analyzer
Analyze water test reports easily
Scan Water Report
OR
Enter Values Manually


20:53
94%
Scan Water Report
Camera
Gallery
Uploaded Image
Water Analysis Result

6.2 Output Design



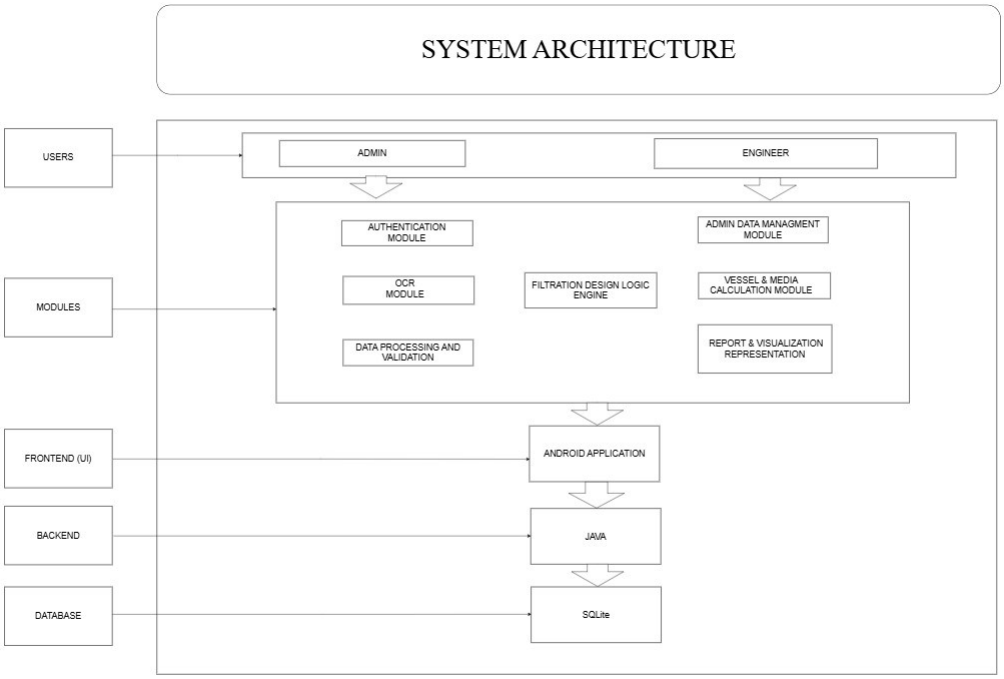




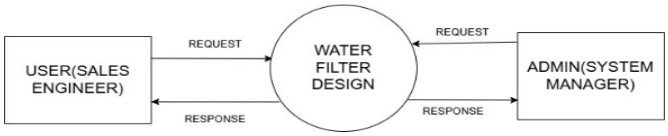
6.3 Architecture Design

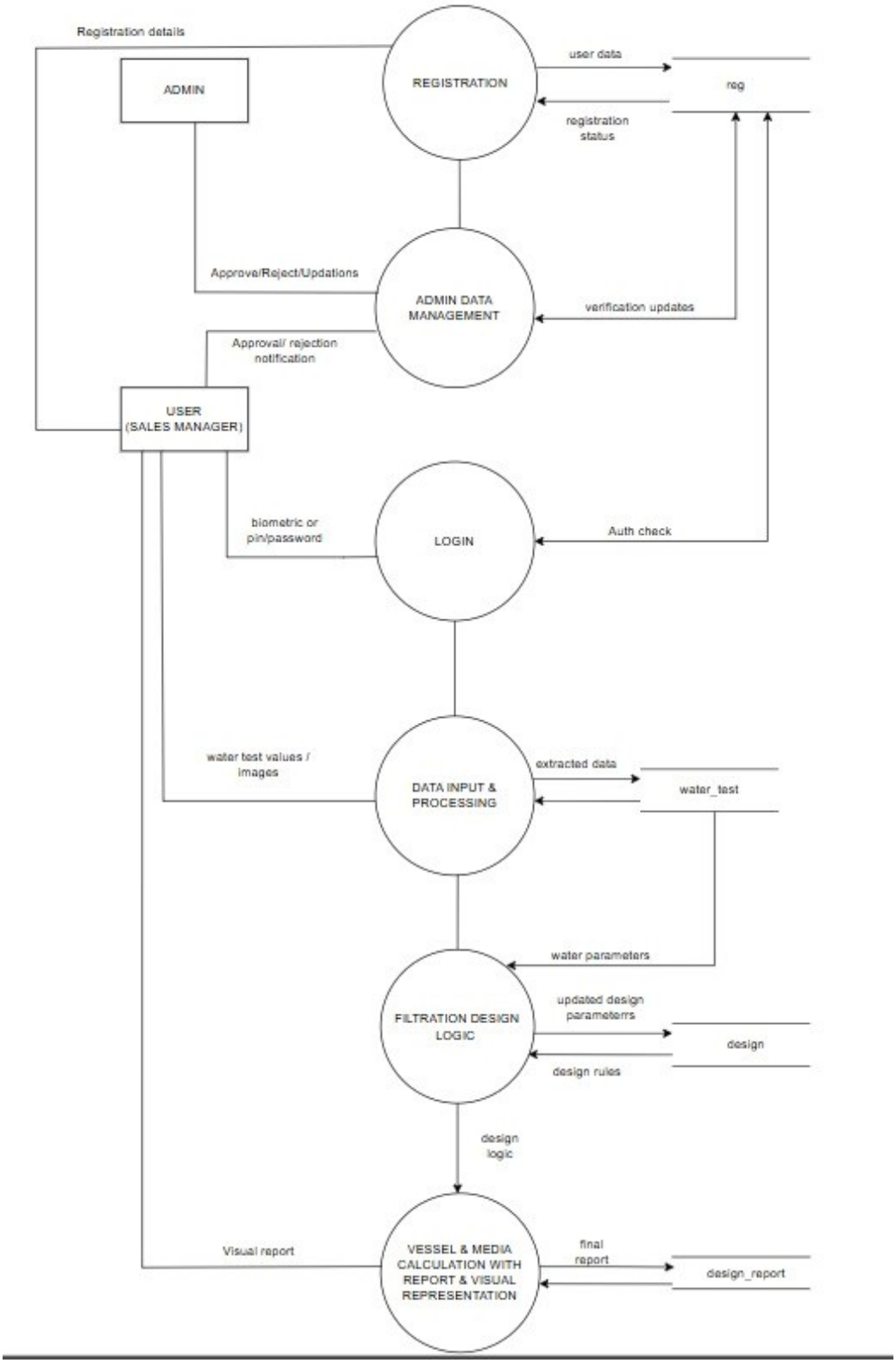
The system is based on a **three-tier architecture**, which separates the application into logical layers:

1. **Presentation Layer (Front-End):** The user interface is built as an Android application, providing interactive screens for users to input and view data.
2. **Application Layer (Back-End):** The core logic of the system is implemented in **Java**, which processes user inputs, performs calculations, and manages the workflow of the application.
3. **Data Layer (Database):** All data is stored and managed in **SQLite**, the lightweight database embedded within the Android application, allowing efficient storage and retrieval of information.

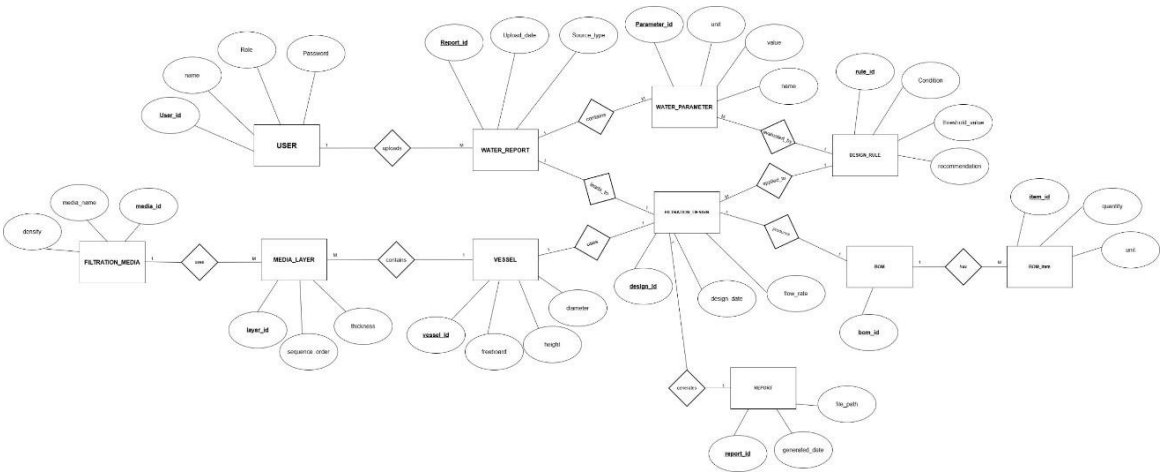


6.3.1 Data Flow Diagram

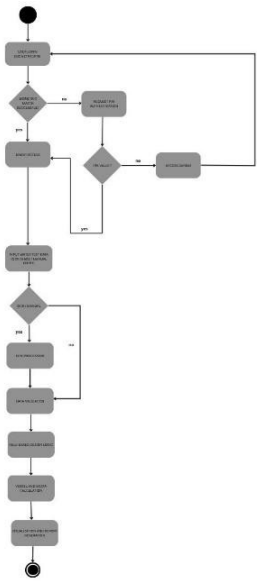



Registration details
ADMIN
REGISTRATION
user data
reg
registration status
Approve/Reject/Updations
ADMIN DATA MANAGEMENT
verification updates
Approval/ rejection notification
USER (SALES MANAGER)
biometric or pin/password
LOGIN
Auth check
water test values / images
DATA INPUT & PROCESSING
extracted data
water_test
water parameters
FILTRATION DESIGN LOGIC
updated design parameterrs
design
design rules
design logic
Visual report
VESSEL & MEDIA CALCULATION WITH REPORT & VISUAL REPRESENTATION
final report
design_report

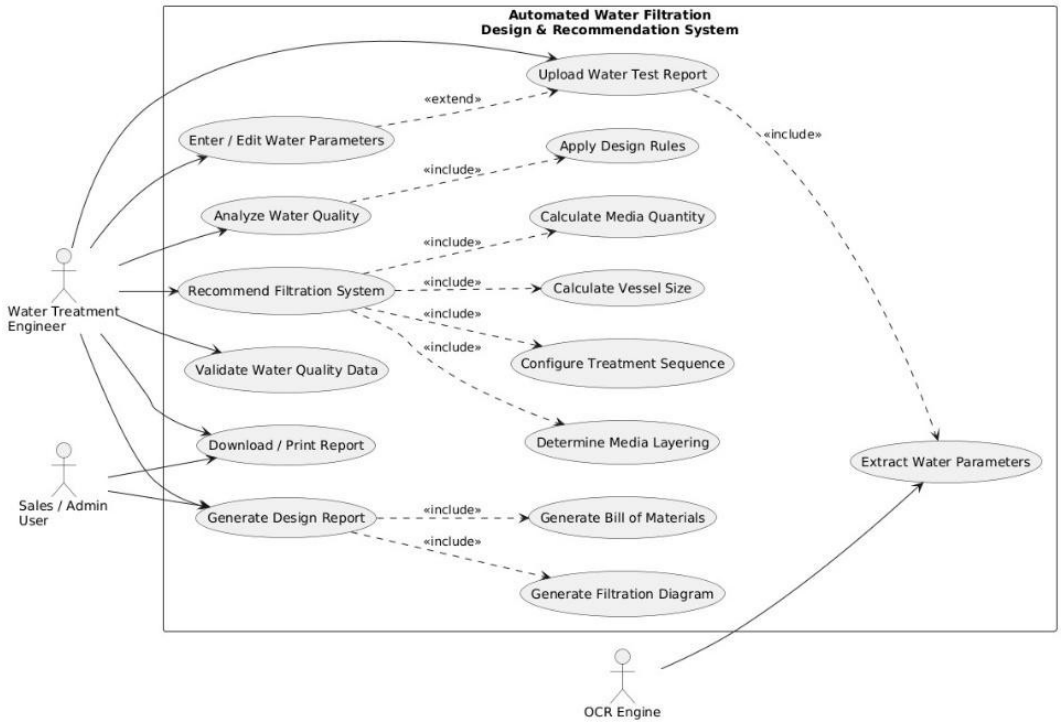
6.3.2 ER Diagram



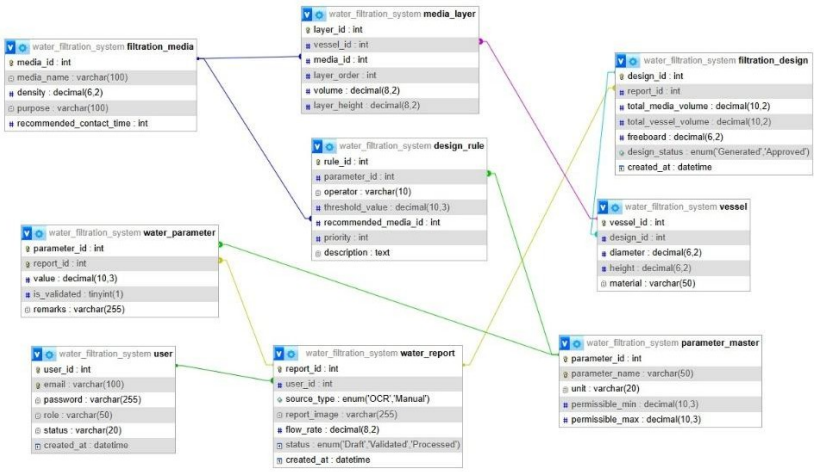
6.3.3 Work Flow Diagram



6.3.4 Use Case Diagram



6.3.5 Table Design



7.0 SYSTEM TESTING AND IMPLEMENTATION

7.1 System Testing

System testing is an essential phase in the software development process, which ensures that the system functions correctly and meets all specified requirements. The testing process is carried out in multiple stages to identify and fix errors at different levels of the application. The main objective of system testing is to ensure accuracy, reliability, and efficiency of the proposed system.

1. **Unit Testing:**

Unit testing involves testing individual components or modules of the system separately to ensure that each part functions correctly. In this system, unit testing is performed on various modules such as the OCR module, input validation module, and calculation module. The OCR module is tested to verify that it accurately extracts values like TDS, hardness, turbidity, and iron from uploaded images. The input forms are tested to ensure that they accept only valid data and properly handle incorrect or missing inputs. The calculation module is tested to confirm that formulas used for media quantity and vessel size produce accurate results.

2. **Integration Testing:**

Integration testing is performed after unit testing to ensure that different modules work together correctly. In this system, integration testing verifies that data flows smoothly from input to processing and then to output. The system is tested to ensure that data entered manually or extracted through OCR is correctly passed to the design engine. It also checks whether the processing module generates correct filtration designs, bill of materials, and vessel configurations. The integration of graphical representation with calculated results is also verified.

3. **User Acceptance Testing (UAT):**

User Acceptance Testing is the final stage of testing, where the system is tested from the user's perspective. In this phase, real-world scenarios are simulated to ensure that the system meets user requirements. Users test the system by entering data, uploading reports, and verifying outputs. The system is checked to ensure that outputs such as filtration designs, BOM, and reports are accurate, clear, and easy to understand. Feedback from users is collected and used to improve the system.

Through this multi-level testing approach, the system ensures high accuracy, smooth functionality, and user satisfaction. It helps in identifying errors early and ensures that the final system is stable, reliable, and ready for deployment.

7.2 Maintenance

Maintenance is an important phase that ensures the system continues to function efficiently after deployment. It involves monitoring the system, fixing issues, updating features, and improving performance based on user feedback and changing requirements.

1. Corrective Maintenance:

Corrective maintenance involves identifying and fixing errors or bugs that occur during system operation. Issues such as incorrect calculations, OCR misinterpretation of data, or system crashes are detected and resolved to maintain system reliability.

2. Adaptive Maintenance:

Adaptive maintenance is performed to update the system according to new requirements or changes in the environment. This includes adding new water quality parameters, updating filtration rules based on industry standards, and ensuring compatibility with updated Android versions and devices.

3. Perfective Maintenance:

Perfective maintenance focuses on improving system performance and usability. Enhancements such as improving user interface design, increasing OCR accuracy, optimizing processing speed, and adding new features are carried out based on user feedback.

4. Preventive Maintenance:

Preventive maintenance is carried out to avoid future problems and ensure long-term stability of the system. Regular system checks, code optimization, database maintenance, and security improvements are performed to prevent failures.

5. Database Maintenance:

The SQLite database is regularly maintained to ensure data integrity and prevent data loss. Backup procedures are implemented to safeguard important data and ensure smooth system operation.

6. Performance Monitoring:

The system is continuously monitored to ensure it performs efficiently under different conditions. This includes checking processing speed, accuracy of results, and user experience during operation.

Overall, maintenance ensures that the system remains reliable, secure, and efficient over time. It allows the system to adapt to future requirements and provides long-term usability.

CONCLUSION

8.0 CONCLUSION

The **Automated Water Filtration Design and Recommendation System** has been successfully developed to overcome the limitations of the traditional manual approach used in designing water filtration systems. In the existing system, engineers are required to manually analyze water quality reports, interpret multiple parameters, perform complex calculations, and prepare reports, which is both time-consuming and prone to errors. This project introduces an automated solution that simplifies and standardizes the entire process, making it more efficient and reliable.

The system enables users to input water quality data either manually or through OCR technology, significantly reducing the effort required for data entry. It processes important parameters such as TDS, hardness, turbidity, iron, chloride, pH, and hydrogen sulfide using predefined engineering rules and logic. Based on this analysis, the system automatically selects suitable filtration media, calculates the required media quantity, determines the appropriate vessel size, and generates a complete filtration system design.

In addition to calculations, the system provides useful outputs such as a Bill of Materials (BOM), visual representation of the filtration system, and a detailed report in PDF format. These outputs improve clarity and help users understand the system design easily. By integrating technologies such as Android Studio, Java, SQLite, and Google ML Kit (OCR), the system ensures smooth performance, fast processing, and accurate results.

The automation of the design process reduces human errors, saves time, and ensures consistency, as the same input data produces the same output. It also reduces dependency on experienced engineers, making the system accessible to a wider range of users. The system improves productivity and provides a user-friendly platform for designing water filtration systems.

Overall, the project provides a practical and efficient solution for water filtration system design. It enhances accuracy, reliability, and usability while reducing manual workload. The system serves as a strong foundation for future improvements and has the potential to be used in real-world applications for water treatment and management.

9.0 SCOPE FOR FURTHER DEVELOPMENT

The current water filtration design system provides a reliable and efficient solution for automating the design process. However, its functionality can be further enhanced to create a more advanced, intelligent, and scalable system. The following improvements can be considered for future development:

1. **Integration with IoT Sensors:**
   The system can be connected with real-time water quality sensors to automatically collect parameters such as TDS, pH, turbidity, and hardness. This integration will eliminate manual data entry and enable continuous monitoring of water quality. It will also allow the system to generate filtration designs dynamically based on real-time data.

2. **AI-Based Recommendation System:**
   Artificial Intelligence and Machine Learning techniques can be incorporated to provide smarter and more accurate filtration recommendations. By analyzing historical data, usage patterns, and environmental conditions, the system can suggest optimized and cost-effective filtration solutions.

3. **Mobile Application Enhancements:**
   The application can be further improved by adding features such as offline access, push notifications, and enhanced user interface design. These improvements will increase usability and provide a better user experience for engineers and technicians.

4. **Advanced Visualization:**
   The system can be enhanced to provide detailed and interactive 3D visual representations of filtration systems. This will help users better understand the structure, arrangement, and working of the filtration system, making it more effective for presentation and implementation.

5. **Cloud Integration:**
   The system can be extended to a cloud-based platform, allowing users to access data from multiple devices and locations. This will also enable secure data storage, backup, and sharing.

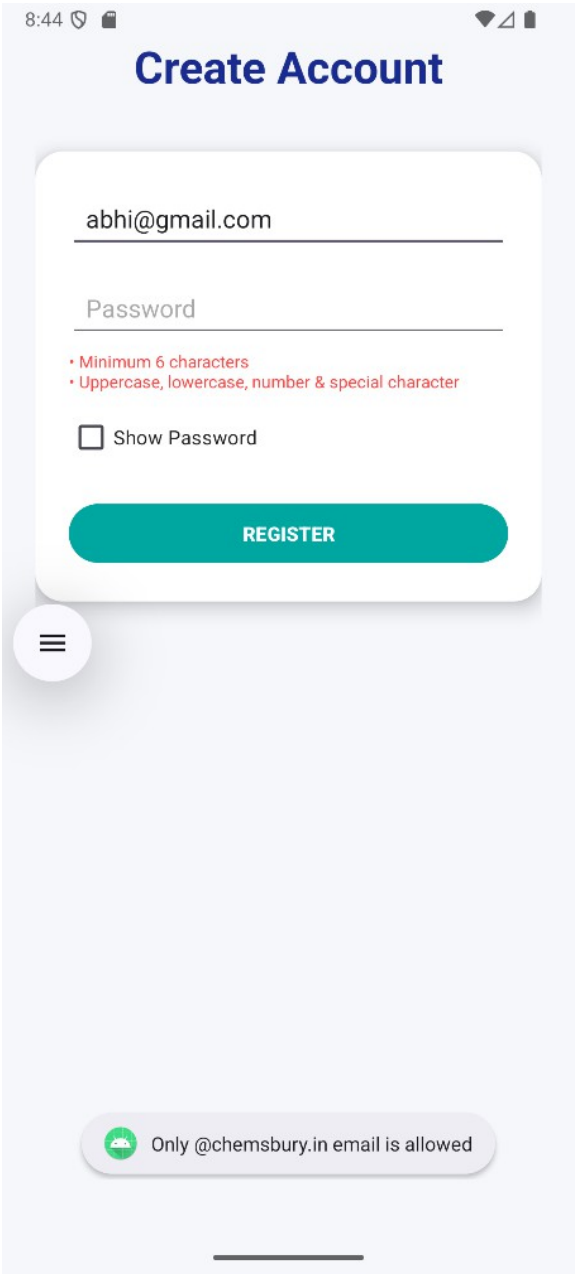
6. **Cost Estimation Module:**
   A cost calculation feature can be added to estimate the total cost of materials, installation, and maintenance. This will be useful for commercial applications and quotation generation.

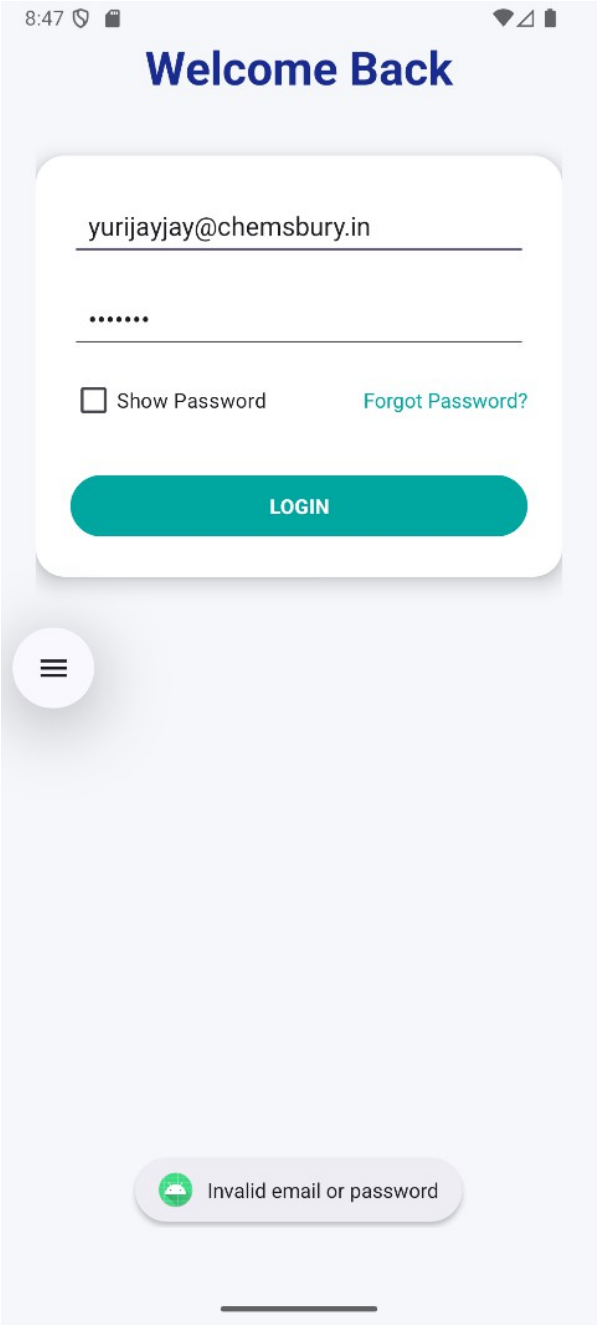
10.0 BIBILIOGRAPHY

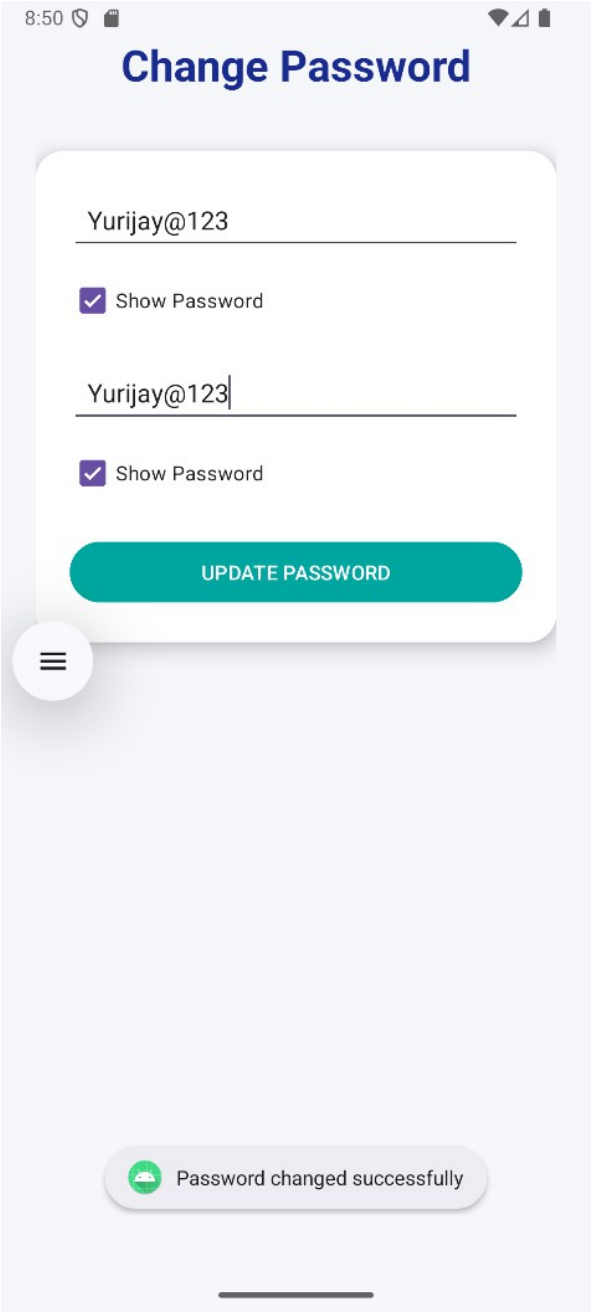
- MWH, *Water Treatment: Principles and Design*, John Wiley & Sons, 2012.
- Metcalf & Eddy, *Wastewater Engineering: Treatment and Resource Recovery*, McGraw-Hill Education, 2014.
- American Water Works Association (AWWA), *Water Quality and Treatment: A Handbook on Drinking Water*, McGraw-Hill.
- World Health Organization (WHO), *Guidelines for Drinking-Water Quality*, WHO Press.
- Google, *ML Kit Documentation*. Available at: https://developers.google.com/ml-kit
- Google, *Android Developer Documentation*, Available at: https://developer.android.com
- SQLite Consortium, *SQLite Documentation*, Available at: https://www.sqlite.org/docs.html
- Tesseract OCR, *Open Source Optical Character Recognition Engine*, Available at: https://github.com/tesseract-ocr
- Research papers on water filtration systems, OCR technology, and mobile application development from IEEE and other journals.
- Online technical articles and tutorials related to Android development, database management, and OCR implementation.

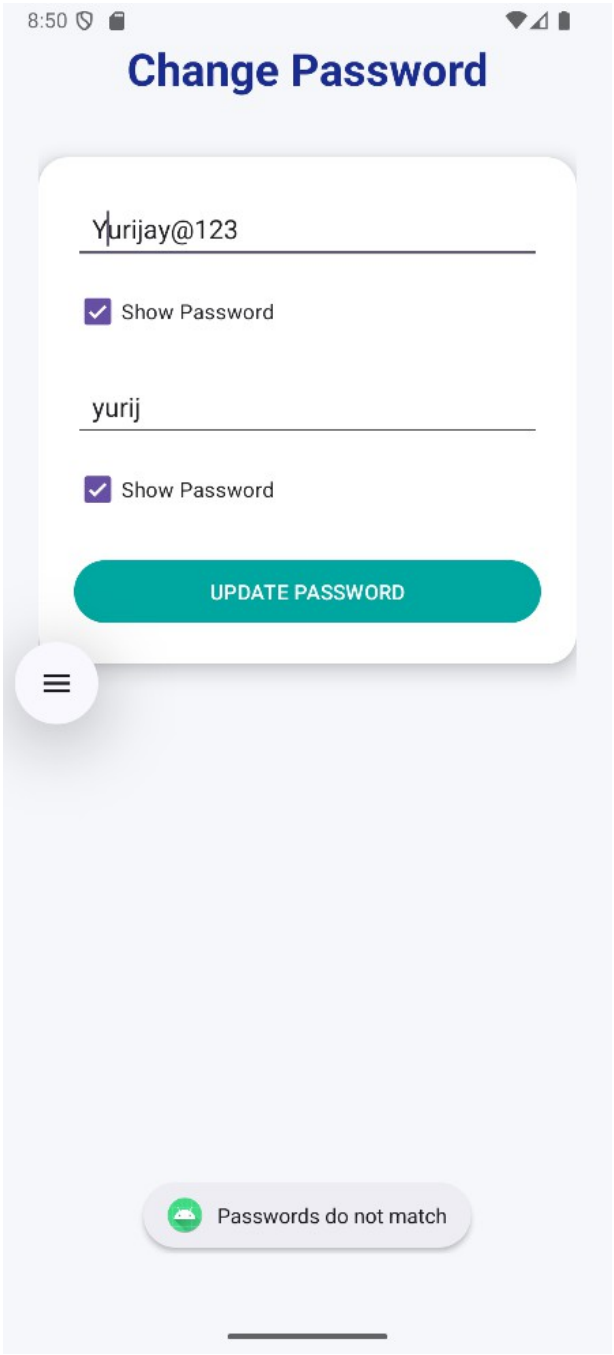
11.0 APPENDIX

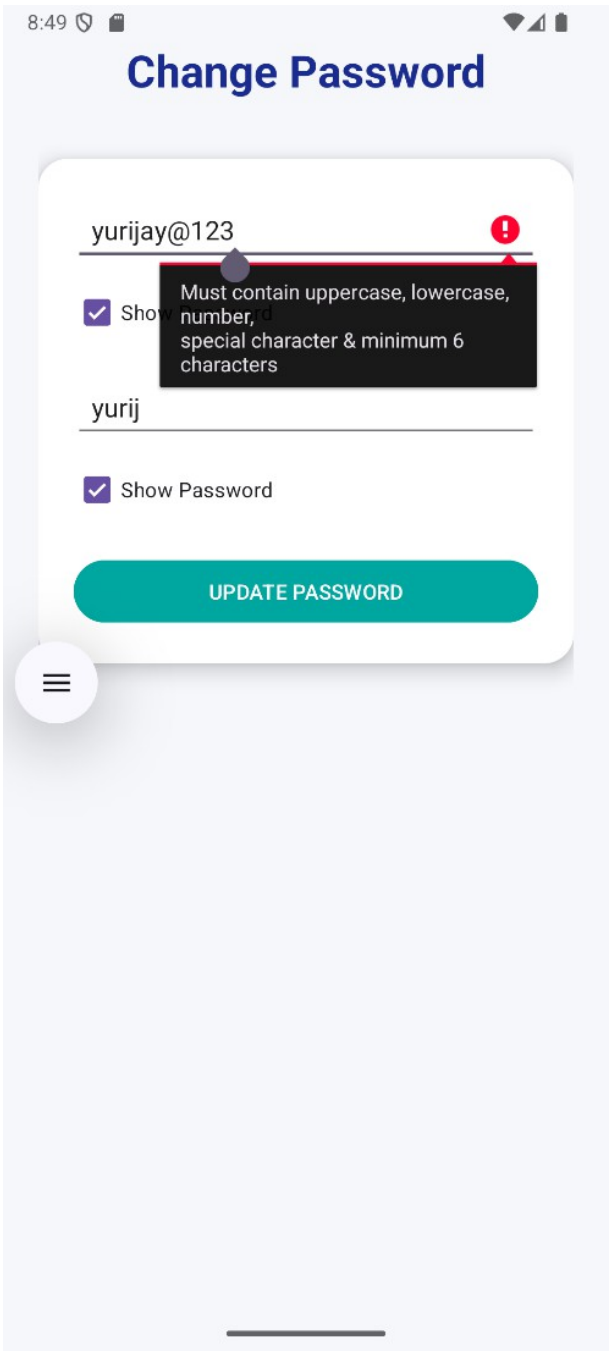
11.1 Screen Shots

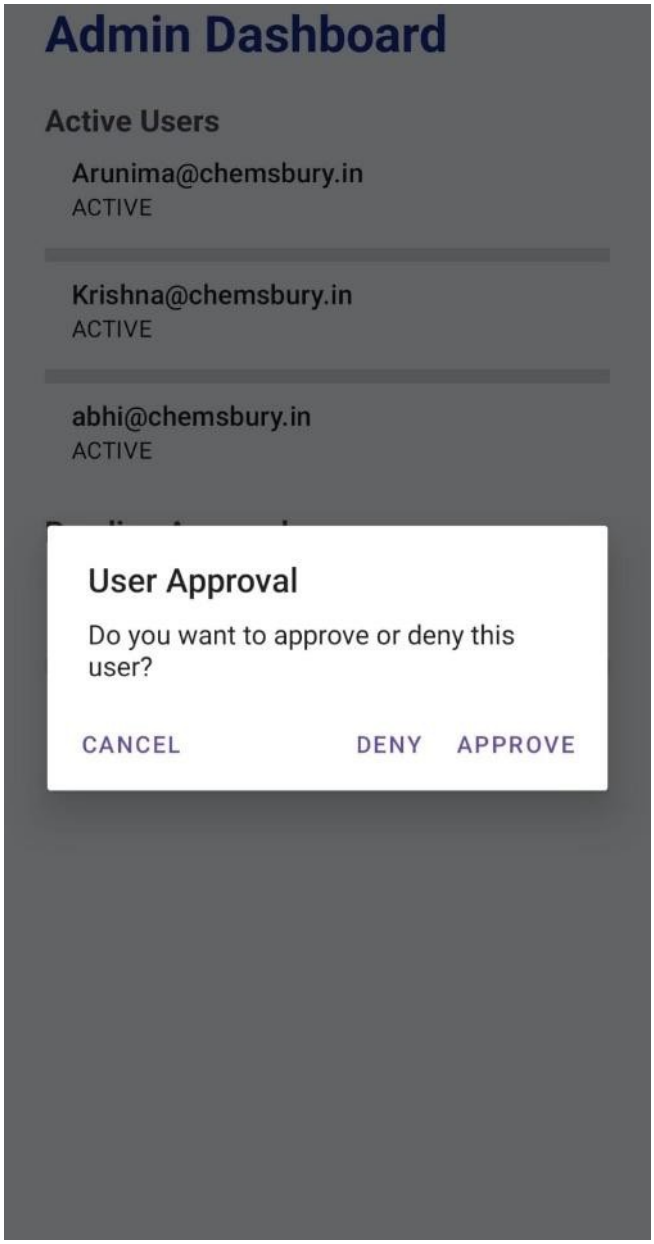


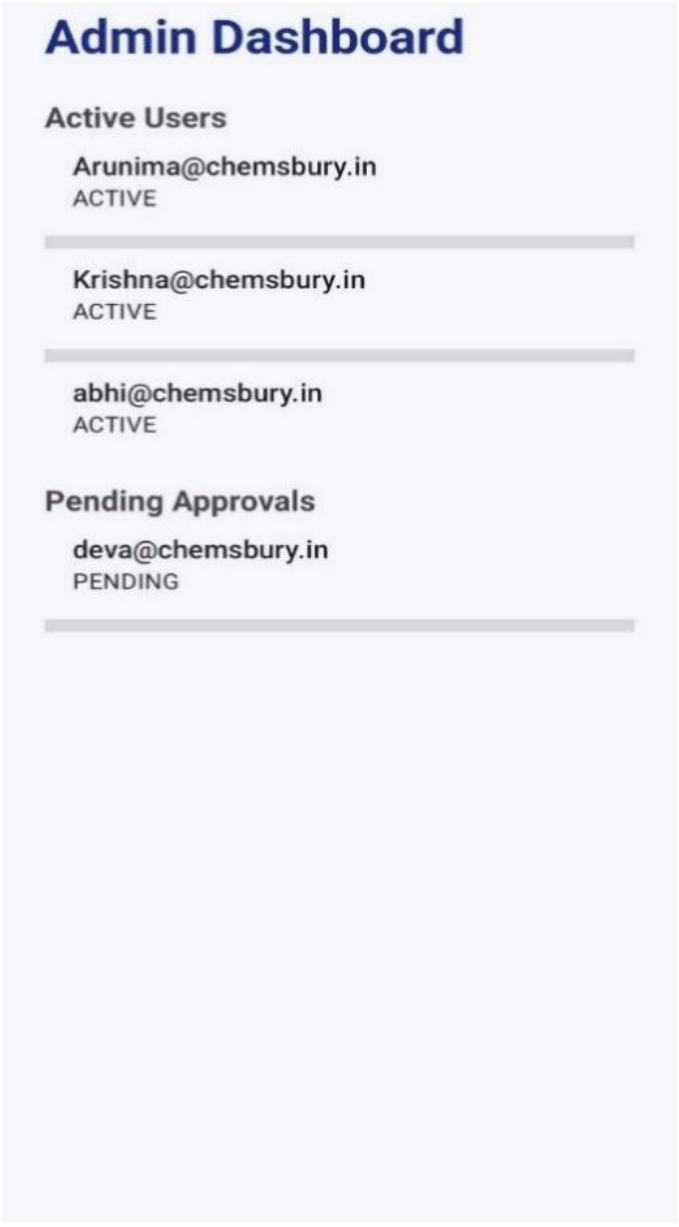

8:47
Welcome Back
yurijayjay@chemsbury.in
Show Password
Forgot Password?
LOGIN
Invalid email or password


8:50
Change Password
Yurijay@123
Show Password
Yurijay@123
Show Password
UPDATE PASSWORD
Password changed successfully


8:50
Change Password
Yurijay@123
Show Password
yurij
Show Password
UPDATE PASSWORD
Passwords do not match

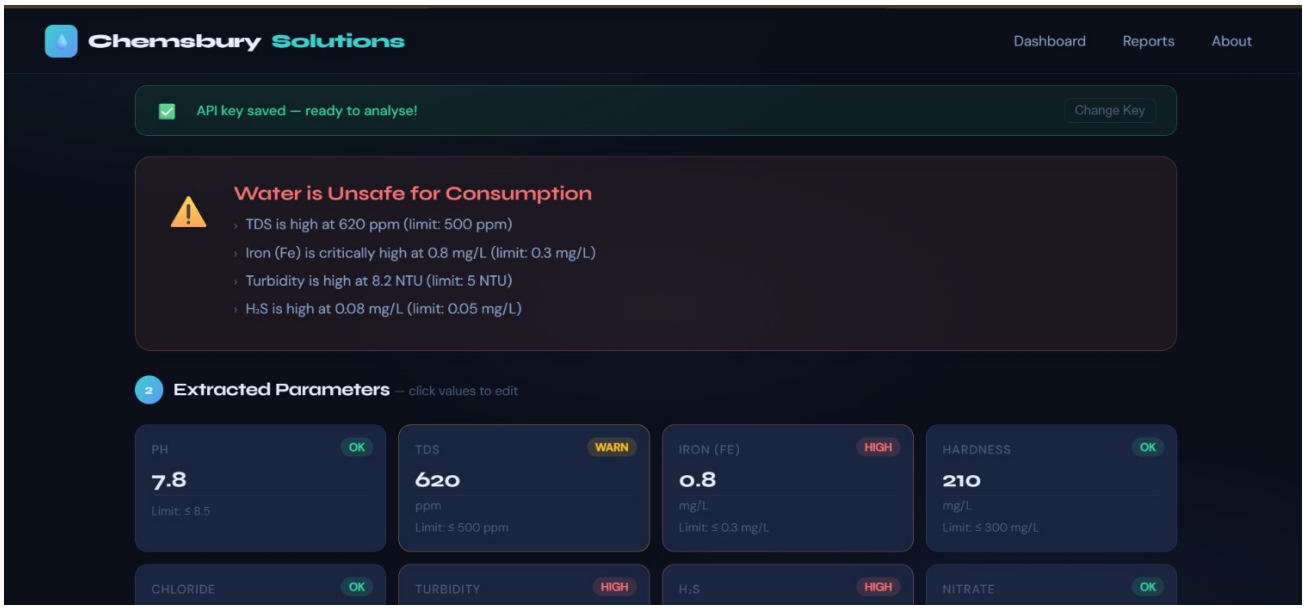

8:49
Change Password
yurijay@123
Must contain uppercase, lowercase, number,
special character & minimum 6 characters
yurij
Show Password
UPDATE PASSWORD

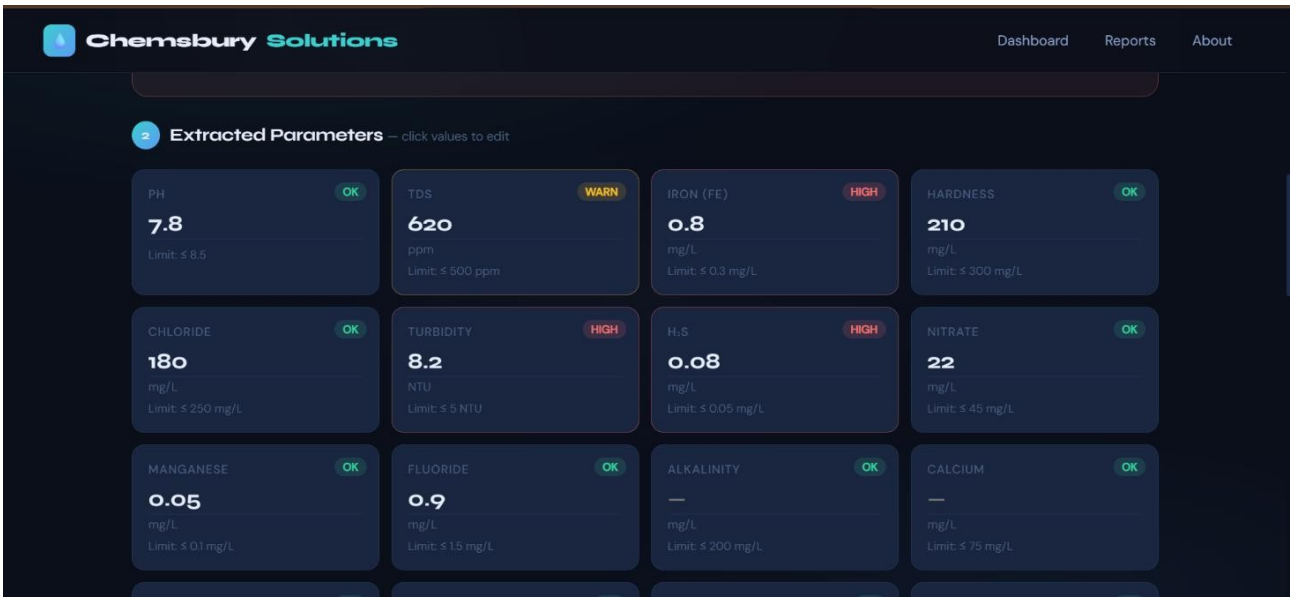

Admin Dashboard
Active Users
Arunima@chemsbury.in
ACTIVE
Krishna@chemsbury.in
ACTIVE
abhi@chemsbury.in
ACTIVE
User Approval
Do you want to approve or deny this user?
CANCEL
DENY
APPROVE

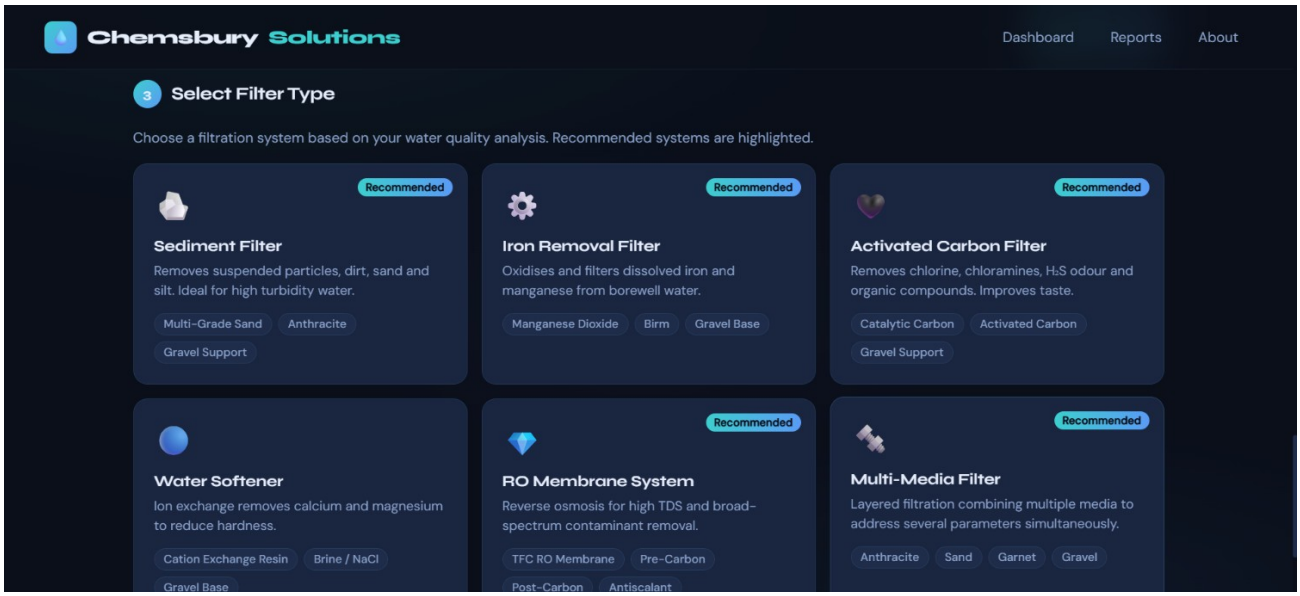

Admin Dashboard
Active Users
Arunima@chemsbury.in
ACTIVE
Krishna@chemsbury.in
ACTIVE
abhi@chemsbury.in
ACTIVE
Pending Approvals
deva@chemsbury.in
PENDING

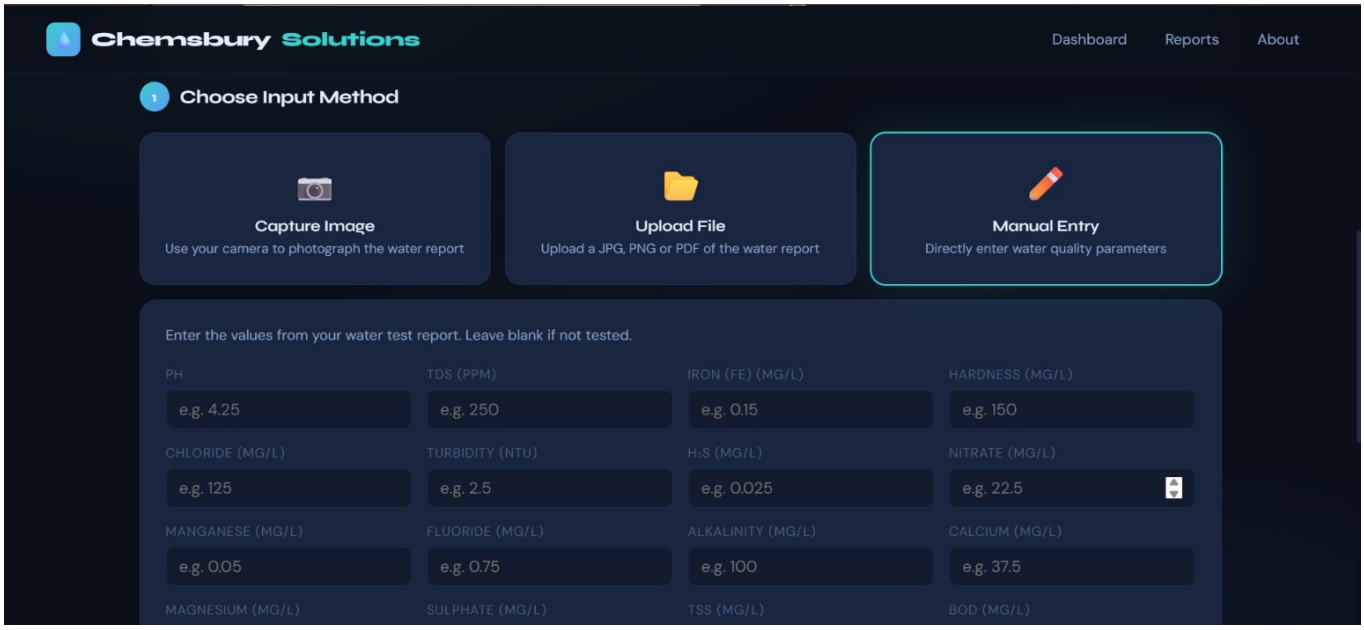
Website

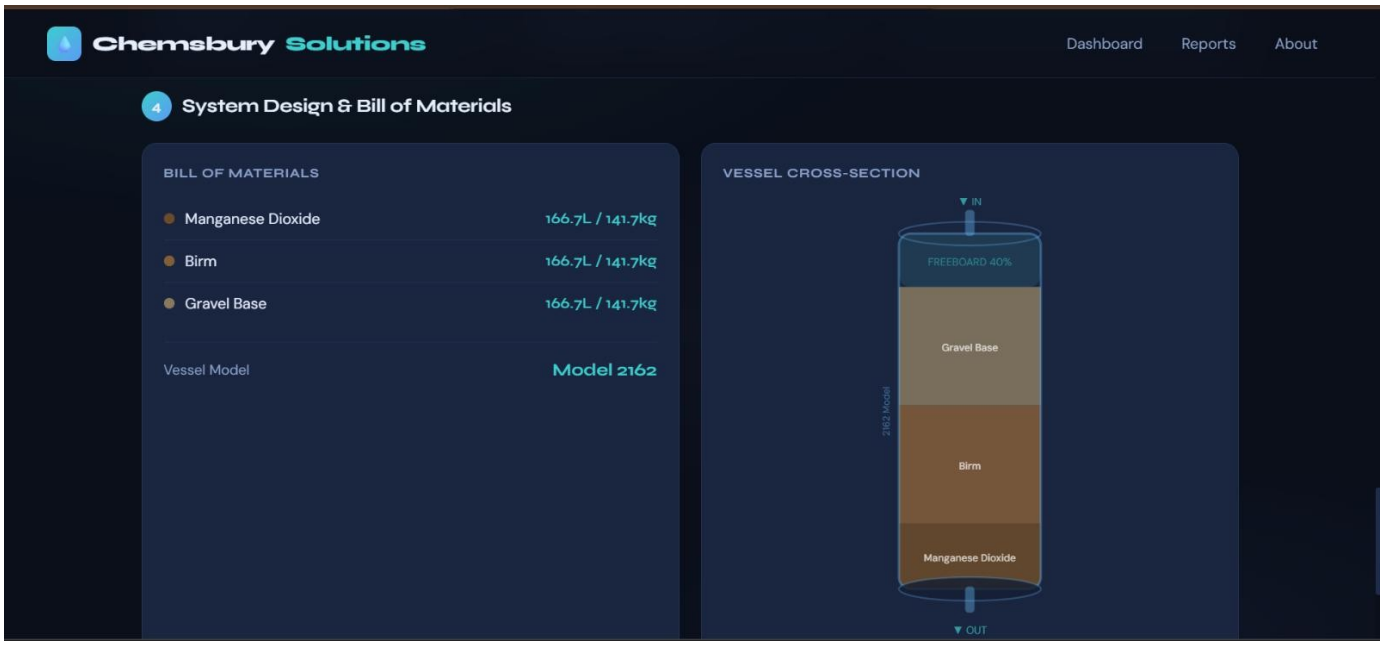

Chemsbury Solutions
Dashboard
Reports
About
AI-POWERED WATER ANALYSIS
Precision Water Quality Diagnosis & Filter Design
Upload a water test report or capture one with your camera. Our AI extracts all parameters, assesses safety, and recommends the ideal filtration system for your needs.

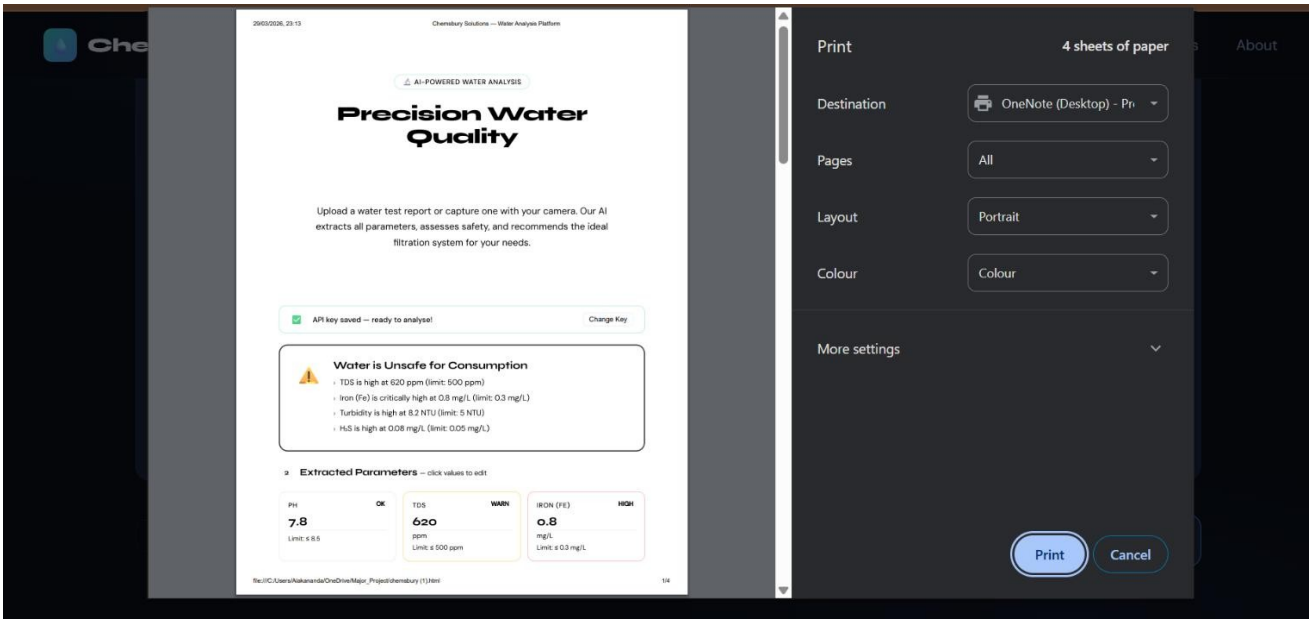

Chemsbury Solutions
Dashboard
Reports
About
API key saved — ready to analyse!
Change Key
Water is Unsafe for Consumption
TDS is high at 620 ppm (limit: 500 ppm)
Iron (Fe) is critically high at 0.8 mg/L (limit: 0.3 mg/L)
Turbidity is high at 8.2 NTU (limit: 5 NTU)
H₂S is high at 0.08 mg/L (limit: 0.05 mg/L)
Extracted Parameters — click values to edit
PH OK 7.8 Limit: ≤ 8.5
TDS WARN 620 ppm Limit: ≤ 500 ppm
IRON (FE) HIGH 0.8 mg/L Limit: ≤ 0.3 mg/L
HARDNESS OK 210 mg/L Limit: ≤ 300 mg/L
CHLORIDE OK
TURBIDITY HIGH
H₂S HIGH
NITRATE OK


Chemsbury Solutions
Dashboard
Reports
About
2 Extracted Parameters — click values to edit
PH
OK
7.8
Limit: ≤ 8.5
TDS
WARN
620
ppm
Limit: ≤ 500 ppm
IRON (FE)
HIGH
0.8
mg/L
Limit: ≤ 0.3 mg/L
HARDNESS
OK
210
mg/L
Limit: ≤ 300 mg/L
CHLORIDE
OK
180
mg/L
Limit: ≤ 250 mg/L
TURBIDITY
HIGH
8.2
NTU
Limit: ≤ 5 NTU
H₂S
HIGH
0.08
mg/L
Limit: ≤ 0.05 mg/L
NITRATE
OK
22
mg/L
Limit: ≤ 45 mg/L
MANGANESE
OK
0.05
mg/L
Limit: ≤ 0.1 mg/L
FLUORIDE
OK
0.9
mg/L
Limit: ≤ 1.5 mg/L
ALKALINITY
OK
—
mg/L
Limit: ≤ 200 mg/L
CALCIUM
OK
—
mg/L
Limit: ≤ 75 mg/L


Chemsbury Solutions
Dashboard
Reports
About
3 Select Filter Type
Choose a filtration system based on your water quality analysis. Recommended systems are highlighted.
Recommended
Sediment Filter
Removes suspended particles, dirt, sand and silt. Ideal for high turbidity water.
Multi-Grade Sand
Anthracite
Gravel Support
Recommended
Iron Removal Filter
Oxidises and filters dissolved iron and manganese from borewell water.
Manganese Dioxide
Birm
Gravel Base
Recommended
Activated Carbon Filter
Removes chlorine, chloramines, H₂S odour and organic compounds. Improves taste.
Catalytic Carbon
Activated Carbon
Gravel Support
Water Softener
Ion exchange removes calcium and magnesium to reduce hardness.
Cation Exchange Resin
Brine / NaCl
Gravel Base
Recommended
RO Membrane System
Reverse osmosis for high TDS and broad-spectrum contaminant removal.
TFC RO Membrane
Pre-Carbon
Post-Carbon
Antiscalant
Recommended
Multi-Media Filter
Layered filtration combining multiple media to address several parameters simultaneously.
Anthracite
Sand
Garnet
Gravel


Chemsbury Solutions
Dashboard
Reports
About
1 Choose Input Method
Capture Image
Use your camera to photograph the water report
Upload File
Upload a JPG, PNG or PDF of the water report
Manual Entry
Directly enter water quality parameters
Enter the values from your water test report. Leave blank if not tested.
PH
e.g. 4.25
TDS (PPM)
e.g. 250
IRON (FE) (MG/L)
e.g. 0.15
HARDNESS (MG/L)
e.g. 150
CHLORIDE (MG/L)
e.g. 125
TURBIDITY (NTU)
e.g. 2.5
H₂S (MG/L)
e.g. 0.025
NITRATE (MG/L)
e.g. 22.5
MANGANESE (MG/L)
e.g. 0.05
FLUORIDE (MG/L)
e.g. 0.75
ALKALINITY (MG/L)
e.g. 100
CALCIUM (MG/L)
e.g. 37.5
MAGNESIUM (MG/L)
SULPHATE (MG/L)
TSS (MG/L)
BOD (MG/L)


Chemsbury Solutions
Dashboard
Reports
About
4 System Design & Bill of Materials
BILL OF MATERIALS
Manganese Dioxide
166.7L / 141.7kg
Birm
166.7L / 141.7kg
Gravel Base
166.7L / 141.7kg
Vessel Model
Model 2162
VESSEL CROSS-SECTION
▼ IN
FREEBOARD 40%
Gravel Base
2162 Model
Birm
Manganese Dioxide
▼ OUT


Che
About
20/03/2026, 23:13
Chemsbury Solutions — Water Analysis Platform
AI-POWERED WATER ANALYSIS
Precision Water Quality
Upload a water test report or capture one with your camera. Our AI extracts all parameters, assesses safety, and recommends the ideal filtration system for your needs.
API key saved — ready to analyse!
Change Key
Water is Unsafe for Consumption
TDS is high at 620 ppm (limit: 500 ppm)
Iron (Fe) is critically high at 0.8 mg/L (limit: 0.3 mg/L)
Turbidity is high at 8.2 NTU (limit: 5 NTU)
H₂S is high at 0.08 mg/L (limit: 0.05 mg/L)
Extracted Parameters – click values to edit
PH
OK
7.8
Limit: ≤ 8.5
TDS
WARN
620
ppm
Limit: ≤ 500 ppm
IRON (FE)
HIGH
0.8
mg/L
Limit: ≤ 0.3 mg/L
file:///C:/Users/Aakananda/OneDrive/Major_Project/chemsbury (1).html
1/4
Print
4 sheets of paper
Destination
OneNote (Desktop) - Pr
Pages
All
Layout
Portrait
Colour
Colour
More settings
Print
Cancel

11.2 Sample Code

Register.java

```
package com.example.waterfilter;
import android.os.Bundle;
import android.text.method.HideReturnsTransformationMethod;
import android.text.method.PasswordTransformationMethod;
import android.util.Patterns;
import android.view.View;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.EditText;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class Register extends AppCompatActivity
   { EditText email,pass;
   Button reg;
   CheckBox showPassword;
   DatabaseHelper dbHelper;


   @Override
   protected void onCreate(Bundle savedInstanceState)
      { super.onCreate(savedInstanceState);
      EdgeToEdge.enable(this);
      setContentView(R.layout.activity_register);
      email=findViewById(R.id.editEmail);
```

```
        pass=findViewById(R.id.editPassword);
        reg=findViewById(R.id.buttonRegister);
        showPassword = findViewById(R.id.showPasswordCheckBox);
        dbHelper = new DatabaseHelper(this);
        showPassword.setOnCheckedChangeListener((buttonView, isChecked) ->
           { if (isChecked) {

pass.setTransformationMethod(HideReturnsTransformationMethod.getInstance());
           } else {

pass.setTransformationMethod(PasswordTransformationMethod.getInstance());
           }
           pass.setSelection(pass.getText().length());
        });
        reg.setOnClickListener(new View.OnClickListener()
           { @Override
           public void onClick(View v) {
              String userEmail=email.getText().toString().trim();
              String userPass=pass.getText().toString().trim();
              if(userEmail==null || userEmail.isEmpty()){
                 Toast.makeText(Register.this, "Email and Password cannot be empty",
Toast.LENGTH_SHORT).show();
                 return;



              }
              // Accepts gmail.com and chemsbury.in only
              String emailPattern = "^[a-zA-Z0-9._%+-]+@chemsbury\\.in$";

              if (!userEmail.matches(emailPattern))
                 { Toast.makeText(
                       Register.this,
                       "Only @chemsbury.in email is allowed",
                       Toast.LENGTH_LONG
```

```
            ).show();
            return;
        }



        //Strong validation
        String passwordPattern = "^(?=.*[a-z])(?=.*[A-
Z])(?=.*\\d)(?=.*[@$!%*?&])[A-Za-z\\d@$!%*?&]{6,}$";

        if (!userPass.matches(passwordPattern)) {
            Toast.makeText(Register.this, "Invalid password",
Toast.LENGTH_SHORT).show();
            return;
        }

        if (dbHelper.checkEmailExists(userEmail)) {
            Toast.makeText(Register.this, "Email already registered",
Toast.LENGTH_SHORT).show();
            return;
        }

        boolean inserted = dbHelper.insertUser(userEmail, userPass);
        if (inserted) {
            Toast.makeText(Register.this, "Registration Successful",
Toast.LENGTH_SHORT).show();
            email.setText("");
            pass.setText("");
            showPassword.setChecked(false);
        } else {
            Toast.makeText(Register.this, "Registration Failed",
Toast.LENGTH_SHORT).show();}
```

```
        }
    });

}}
```

register.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#F5F7FB"
    android:padding="24dp">

    <TextView
        android:id="@+id/textTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Create Account"
        android:textSize="32sp"
        android:textStyle="bold"
        android:textColor="#1A2C90"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>
    <androidx.cardview.widget.CardView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        app:cardCornerRadius="20dp"
        app:cardElevation="8dp"
        android:layout_marginTop="40dp"
```

```
        app:layout_constraintTop_toBottomOf="@id/textTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">
        <LinearLayout
            android:orientation="vertical"
            android:padding="24dp"
            android:layout_width="match_parent"
            android:layout_height="wrap_content">
            <EditText
                android:id="@+id/editEmail"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="Company Email"
                android:inputType="textEmailAddress"
                android:padding="12dp"/>
            <EditText
                android:id="@+id/editPassword"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="Password"
                android:layout_marginTop="16dp"
                android:inputType="textPassword|textNoSuggestions"
                android:padding="12dp"/>
            <TextView
                android:id="@+id/passwordRules"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="• Minimum 6 characters\n• Uppercase, lowercase, number
&amp;amp; special character"
                android:textSize="12sp"
                android:textColor="#FF4444"
                android:layout_marginTop="6dp"/>
            <CheckBox
                android:id="@+id/showPasswordCheckBox"
```

```
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Show Password"
            android:layout_marginTop="8dp"/>
        <Button
            android:id="@+id/buttonRegister"
            android:layout_width="match_parent"
            android:layout_height="50dp"
            android:text="REGISTER"
            android:textStyle="bold"
            android:textColor="@android:color/white"
            android:layout_marginTop="20dp"
            android:backgroundTint="#00A7A0"/>
    </LinearLayout>
  </androidx.cardview.widget.CardView>
</androidx.constraintlayout.widget.ConstraintLayout>
```

Login.java

```
package com.example.waterfilter;
import android.content.Intent;
import android.os.Bundle;
import android.text.method.HideReturnsTransformationMethod;
import android.text.method.PasswordTransformationMethod;
import android.view.View;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import com.example.waterfilter.DatabaseHelper;
```

```
public class Login extends AppCompatActivity
   { EditText editEmail, editPassword;
   Button buttonLogin;
   TextView buttonforgetpass;
   CheckBox showPassword;
   DatabaseHelper db;
   @Override
   protected void onCreate(Bundle savedInstanceState)
      { super.onCreate(savedInstanceState);
      EdgeToEdge.enable(this);
      setContentView(R.layout.activity_login);
      db = new DatabaseHelper(this);
      editEmail = findViewById(R.id.editEmail);
      editPassword = findViewById(R.id.editPassword);
      buttonLogin = findViewById(R.id.buttonLogin);
      showPassword = findViewById(R.id.showPasswordCheckBox);
      buttonforgetpass=findViewById(R.id.buttonForgotPassword);
      // Show/hide password
      showPassword.setOnCheckedChangeListener((buttonView, isChecked) -> {
         if (isChecked)
            { editPassword.setTransformationMethod(
                  HideReturnsTransformationMethod.getInstance()
            );
         } else {
            editPassword.setTransformationMethod( PasswordTransformationMethod.getI
                  nstance()
            );
         }
         editPassword.setSelection(editPassword.getText().length());
      });
      // Login button click
      buttonLogin.setOnClickListener(v -> {

         String email = editEmail.getText().toString().trim();
```

```
        String password = editPassword.getText().toString().trim();

        if (email.isEmpty() || password.isEmpty())
           { Toast.makeText(this, "Email & Password required",
Toast.LENGTH_SHORT).show();
           return;
        }

        // ADMIN LOGIN
        if (db.isAdminLogin(email, password))
           { Toast.makeText(this, "Admin Login Success",
Toast.LENGTH_SHORT).show();
           startActivity(new Intent(Login.this, AdminDashboard.class));
           finish();
           return;
        }

        // USER LOGIN
        if (db.isUserLogin(email, password)) {
           Toast.makeText(this, "User Login Success", Toast.LENGTH_SHORT).show();
           startActivity(new Intent(Login.this, UserDashboard.class));
           finish();
        }
        // USER PENDING
        else if (db.isUserPending(email))
           { Toast.makeText(this,
                "Account not approved yet. Please wait for admin approval.",
                Toast.LENGTH_LONG
           ).show();
        } else {
           Toast.makeText(this, "Invalid email or password",
Toast.LENGTH_LONG).show();
        }
      });
```

```
        buttonforgetpass.setOnClickListener(new View.OnClickListener()
            { @Override
            public void onClick(View v) {
                startActivity(new Intent(Login.this, ForgetPassword.class));
            }
        });
    }
}
```

login.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#F5F7FB"
    android:padding="24dp">

    <!-- TITLE -->
    <TextView
        android:id="@+id/textTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome Back"
        android:textSize="32sp"
        android:textStyle="bold"
        android:textColor="#1A2C90"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>
```

```
<!-- CARD CONTAINER -->
<androidx.cardview.widget.CardView
    android:id="@+id/loginCard"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    app:cardCornerRadius="20dp"
    app:cardElevation="8dp"
    android:layout_marginTop="40dp"
    app:layout_constraintTop_toBottomOf="@id/textTitle"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="24dp">

        <!-- EMAIL -->
        <EditText
            android:id="@+id/editEmail"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Email"
            android:inputType="textEmailAddress"
            android:padding="12dp"/>

        <!-- PASSWORD -->
        <EditText
            android:id="@+id/editPassword"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
```

```
    android:hint="Password"
    android:inputType="textPassword"
    android:padding="12dp"/>

<!-- SHOW PASSWORD -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginTop="8dp"
    android:gravity="center_vertical">
    <CheckBox
        android:id="@+id/showPasswordCheckBox"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Show Password"/>
    <Space
        android:layout_width="0dp"
        android:layout_height="1dp"
        android:layout_weight="1"/>
    <TextView
        android:id="@+id/buttonForgotPassword"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Forgot Password?"
        android:textColor="#00A7A0"/>
</LinearLayout>

<!-- LOGIN BUTTON -->
<Button
    android:id="@+id/buttonLogin"
    android:layout_width="match_parent"
    android:layout_height="50dp"
    android:text="LOGIN"
```

```
            android:textColor="@android:color/white"
            android:textStyle="bold"
            android:layout_marginTop="24dp"
            android:backgroundTint="#00A7A0"/>
    </LinearLayout>
  </androidx.cardview.widget.CardView>
</androidx.constraintlayout.widget.ConstraintLayout>
```

AdminDashboard.java

```
package com.example.waterfilter;
import android.app.AlertDialog;
import android.database.Cursor;
import android.database.MatrixCursor;
import android.os.Bundle;
import android.widget.ListView;
import android.widget.SimpleCursorAdapter;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class AdminDashboard extends AppCompatActivity {
    ListView activeList, pendingList;
    DatabaseHelper db;
    @Override
    protected void onCreate(Bundle savedInstanceState)
      { super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_admin_dashboard);
      activeList = findViewById(R.id.activeList);
      pendingList = findViewById(R.id.pendingList);
      db = new DatabaseHelper(this);
      loadUsers();
    }
    private void loadUsers() {
      Cursor cursor = db.getAllUsers(); // already sorted ACTIVE first
```

```
        // Separate cursors for ACTIVE and PENDING
        MatrixCursor activeCursor = new MatrixCursor(new String[]{"_id", "email",
"status"});
        MatrixCursor pendingCursor = new MatrixCursor(new String[]{"_id", "email",
"status"});

        while (cursor.moveToNext()) {
           int id = cursor.getInt(cursor.getColumnIndexOrThrow("_id"));
           String email = cursor.getString(cursor.getColumnIndexOrThrow("email"));
           String status = cursor.getString(cursor.getColumnIndexOrThrow("status"));

           if (status.equals("ACTIVE"))
              { activeCursor.addRow(new Object[]{id, email, status});
           } else if (status.equals("PENDING"))
              { pendingCursor.addRow(new Object[]{id, email, status});
           }
        }
        cursor.close();
        // Adapter for ACTIVE users
        SimpleCursorAdapter activeAdapter = new
              SimpleCursorAdapter( this,
              android.R.layout.simple_list_item_2,
              activeCursor,
              new String[]{"email", "status"},
              new int[]{android.R.id.text1, android.R.id.text2},
              0
        );
        activeList.setAdapter(activeAdapter);
        // Adapter for PENDING users
        SimpleCursorAdapter pendingAdapter = new
              SimpleCursorAdapter( this,
              android.R.layout.simple_list_item_2,
              pendingCursor,
```

```
            new String[]{"email", "status"},
            new int[]{android.R.id.text1, android.R.id.text2},
            0
        );
        pendingList.setAdapter(pendingAdapter);

        // Handle PENDING user clicks (approve/deny)
        pendingList.setOnItemClickListener((parent, view, position, id) -> {
            Cursor c = (Cursor) parent.getItemAtPosition(position);
            String email = c.getString(c.getColumnIndexOrThrow("email"));
            int userId = c.getInt(c.getColumnIndexOrThrow("_id"));
            new AlertDialog.Builder(this)
                    .setTitle("User Approval")
                    .setMessage("Do you want to approve or deny this user?")
                    .setPositiveButton("Approve", (dialog, which) ->
                        { if (db.approveUser(userId)) {
                            Toast.makeText(this, "User Approved (ACTIVE)",
Toast.LENGTH_SHORT).show();
                            loadUsers(); // refresh
                        }
                    })
                    .setNegativeButton("Deny", (dialog, which) ->
                        { if (db.deleteUser(userId)) {
                            Toast.makeText(this, "User Denied (Deleted)",
Toast.LENGTH_SHORT).show();
                            loadUsers(); // refresh
                        }
                    })
                    .setNeutralButton("Cancel", null)
                    .show();
        });
    }
}
```

admin_dashboard.xml

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#F5F7FB"
    android:padding="24dp">

    <TextView
        android:id="@+id/adminTitle"
        android:text="Admin Dashboard"
        android:textSize="30sp"
        android:textStyle="bold"
        android:textColor="#1A2C90"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"/>

    <TextView
        android:id="@+id/activeHeader"
        android:text="Active Users"
        android:textStyle="bold"
        android:textSize="18sp"
        android:layout_marginTop="20dp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toBottomOf="@id/adminTitle"
        app:layout_constraintStart_toStartOf="parent"/>
```

```
    <ListView
        android:id="@+id/activeList"
        android:layout_width="0dp"
        android:layout_height="200dp"
        android:dividerHeight="8dp"
        app:layout_constraintTop_toBottomOf="@id/activeHeader"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

    <TextView
        android:id="@+id/pendingHeader"
        android:text="Pending Approvals"
        android:textStyle="bold"
        android:textSize="18sp"
        android:layout_marginTop="20dp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toBottomOf="@id/activeList"
        app:layout_constraintStart_toStartOf="parent"/>

    <ListView
        android:id="@+id/pendingList"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:dividerHeight="8dp"
        app:layout_constraintTop_toBottomOf="@id/pendingHeader"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

DatabaseHelper.java

```
package com.example.waterfilter;
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
import java.security.MessageDigest;
public class DatabaseHelper extends SQLiteOpenHelper {
   private static final String DATABASE_NAME = "Chemsbury";
   private static final int DATABASE_VERSION = 8;
   private static final String TABLE_USERS = "user";
   private static final String COLUMN_ID = "user_id";
   private static final String COLUMN_EMAIL = "email";
   private static final String COLUMN_PASSWORD = "password";
   private static final String COLUMN_ROLE = "role";       // ADMIN / USER
   private static final String COLUMN_STATUS = "status";  // ACTIVE / PENDING
   private static final String COLUMN_CREATED_AT = "created_at";
   // Default admin (company email only)
   private static final String ADMIN_EMAIL = "admin@chemsbury.in";
   private static final String ADMIN_PASSWORD = "admin123";
   public DatabaseHelper(Context context) {
      super(context, DATABASE_NAME, null, DATABASE_VERSION);
   }
   //create user table
   @Override
   public void onCreate(SQLiteDatabase db)
      { String createTable =
            "CREATE TABLE " + TABLE_USERS + " (" +
                  COLUMN_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +
                  COLUMN_EMAIL + " TEXT UNIQUE, " +
```

```
                COLUMN_PASSWORD + " TEXT, " +
                COLUMN_ROLE + " TEXT, " +
                COLUMN_STATUS + " TEXT DEFAULT 'PENDING', " +
                COLUMN_CREATED_AT + " DATETIME DEFAULT
CURRENT_TIMESTAMP" +
                ")";
        db.execSQL(createTable);
        // Insert default admin account
        insertAdminIfNotExists(db);
    }
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {

        // Create table if it does not exist (safe upgrade)
        db.execSQL(
            "CREATE TABLE IF NOT EXISTS " + TABLE_USERS + " (" +
                COLUMN_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +
                COLUMN_EMAIL + " TEXT UNIQUE, " +
                COLUMN_PASSWORD + " TEXT, " +
                COLUMN_ROLE + " TEXT, " +
                COLUMN_STATUS + " TEXT DEFAULT 'PENDING', " +
                COLUMN_CREATED_AT + " DATETIME DEFAULT
CURRENT_TIMESTAMP" +
                ")"
        );
    }
     //Checks if email is a Chemsbury company email
    private boolean isChemsbury(String email) {
        return email != null &&
            email.matches("^[A-Za-z0-9._%+-]+@chemsbury\\.in$");
    }

    // PASSWORD HASHING
    private String hashPassword(String password) {
```

```
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(password.getBytes("UTF-8"));

        StringBuilder hex = new StringBuilder();
        for (byte b : hash) {
            String h = Integer.toHexString(0xff & b);
            if (h.length() == 1) hex.append('0');
            hex.append(h);
        }
        return hex.toString();

    } catch (Exception e)
        { return null;
    }
}
 // Inserts admin account if not already present
private void insertAdminIfNotExists(SQLiteDatabase db) {

    // Ensure admin email is company email
    if (!isChemsbury(ADMIN_EMAIL)) return;

    Cursor cursor = db.rawQuery(
            "SELECT 1 FROM user WHERE email=?",
            new String[]{ADMIN_EMAIL}
    );
    // If admin does not exist, insert
    if (cursor.getCount() == 0) {
        ContentValues values = new ContentValues();
        values.put(COLUMN_EMAIL, ADMIN_EMAIL);
        values.put(COLUMN_PASSWORD, hashPassword(ADMIN_PASSWORD));
        values.put(COLUMN_ROLE, "ADMIN");
        values.put(COLUMN_STATUS, "ACTIVE");
        db.insert(TABLE_USERS, null, values);
```

```
        }
        cursor.close();
    }

    // ================== USER REGISTRATION ==================
    public boolean insertUser(String email, String password) {
        // Allow ONLY chemsbury.in
        if (!isChemsbury(email)) {
            return false;
        }
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put(COLUMN_EMAIL, email);
        values.put(COLUMN_PASSWORD, hashPassword(password));
        values.put(COLUMN_ROLE, "USER");
        values.put(COLUMN_STATUS, "PENDING"); // admin approval needed
        long result = db.insert(TABLE_USERS, null, values);
        return result != -1;
    }

     // User login (only ACTIVE users allowed)
    public boolean isUserLogin(String email, String password)
        { SQLiteDatabase db = this.getReadableDatabase();
        Cursor cursor = db.rawQuery(
                "SELECT 1 FROM user WHERE email=? AND password=? AND
role='USER' AND status='ACTIVE'",
                new String[]{email, hashPassword(password)}
        );
        boolean ok = cursor.getCount() > 0;
        cursor.close();
        return ok;
    }
    //Admin login
    public boolean isAdminLogin(String email, String password) {
```

```
        SQLiteDatabase db = this.getReadableDatabase();
        Cursor cursor = db.rawQuery(
                "SELECT 1 FROM user WHERE email=? AND password=? AND
role='ADMIN'",
                new String[]{email, hashPassword(password)}
        );
        boolean ok = cursor.getCount() > 0;
        cursor.close();
        return ok;
    }
    //Returns all users (excluding admin)
    public Cursor getAllUsers() {
        SQLiteDatabase db = this.getReadableDatabase();

        return db.rawQuery(
                "SELECT user_id AS _id, email, status FROM user " +
                    "WHERE role='USER' " +
                    "ORDER BY CASE WHEN status='ACTIVE' THEN 1 ELSE 2 END,
email",
                null
        );
    }
     //Approves a user (PENDING -> ACTIVE)

    public boolean approveUser(int userId)
        { SQLiteDatabase db = this.getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put(COLUMN_STATUS, "ACTIVE");
        return
                db.update( TAB
                LE_USERS,
                values,
                "user_id=?",
                new String[]{String.valueOf(userId)}
```

) > 0;

```
    }

    //Deletes a user
    public boolean deleteUser(int userId)
        { SQLiteDatabase db = this.getWritableDatabase();

        return
                db.delete( TABL
                E_USERS,
                "user_id=?",
                new String[]{String.valueOf(userId)}
        ) > 0;
    }

    // Checks if a user account exists but is still pending admin approval
    public boolean isUserPending(String email) {
        SQLiteDatabase db = this.getReadableDatabase();
        Cursor cursor = db.rawQuery(
                "SELECT 1 FROM user WHERE email=? AND role='USER' AND
status='PENDING'",
                new String[]{email}
        );

        boolean pending = cursor.getCount() > 0;
        cursor.close();
        return pending;
    }
     //Checks if email already exists
// ================= EMAIL VERIFICATION =================
     public boolean checkEmailExists(String email)
         { SQLiteDatabase db = this.getReadableDatabase();

         Cursor cursor = db.rawQuery(
                 "SELECT 1 FROM " + TABLE_USERS + " WHERE " + COLUMN_EMAIL
```

+ " = ?",

```
                new String[]{email}
        );
        boolean exists = cursor.moveToFirst();
        cursor.close();

        return exists;
    }
    // UPDATE PASSWORD
    public boolean updatePasswordByEmail(String email, String newPassword)
        { SQLiteDatabase db = this.getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put(COLUMN_PASSWORD, hashPassword(newPassword));
        int rows = db.update(
                TABLE_USERS,
                values,
                COLUMN_EMAIL + " = ?",
                new String[]{email}
        );

        return rows > 0;
    }
}
```

OCRActivity.java

```
package com.example.myapplication;

import android.Manifest;

import android.content.pm.PackageManager;

import android.graphics.Bitmap;
```

```
import android.graphics.BitmapFactory;
import android.graphics.Color;
import android.graphics.pdf.PdfRenderer;
import android.net.Uri;
import android.os.Bundle;
import android.os.ParcelFileDescriptor;
import android.provider.MediaStore;
import android.util.Base64;
import android.view.View;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.ScrollView;
import android.widget.TextView;
import android.widget.Toast;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;
import androidx.core.content.FileProvider;
import com.google.firebase.functions.FirebaseFunctions;
import android.util.Base64;
import java.io.ByteArrayOutputStream;
```

```
import com.google.common.util.concurrent.ListenableFuture;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.InputStream;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.HashMap;
import java.util.Locale;
import java.util.Map;
public class OCRActivity extends AppCompatActivity
   { private ImageView imgPreview;
   private ScrollView scrollViewOCR;
   private TextView tvResult;
   private LinearLayout layoutParameters;
   private Uri cameraImageUri;
   //Camera
   private final ActivityResultLauncher<Uri> cameraLauncher =
         registerForActivityResult(new ActivityResultContracts.TakePicture(), success ->
{
            if (success && cameraImageUri != null)
               { try {
                  Bitmap bitmap =
                        MediaStore.Images.Media.getBitmap( getContentResolver(),
                        cameraImageUri);
                  showPreviewAndProcess(bitmap);
```

} catch (Exception e) {

```
                showError("Camera error: " + e.getMessage());
            }
        }
    });
// Gallery
private final ActivityResultLauncher<String> galleryLauncher =
    registerForActivityResult(new ActivityResultContracts.GetContent(), uri -> {
        if (uri != null)
            { try {
                InputStream is = getContentResolver().openInputStream(uri);
                Bitmap bitmap = BitmapFactory.decodeStream(is);
                showPreviewAndProcess(bitmap);
            } catch (Exception e) {
                showError("Gallery error: " + e.getMessage());
            }
        }
    });
// PDF
private final ActivityResultLauncher<String[]> pdfLauncher =
    registerForActivityResult(new ActivityResultContracts.OpenDocument(), uri -> {
        if (uri != null)
            { try {
                ParcelFileDescriptor pfd =
                        getContentResolver().openFileDescriptor(uri, "r");
```

```
                if (pfd != null) {
                    PdfRenderer renderer = new PdfRenderer(pfd);
                    PdfRenderer.Page page = renderer.openPage(0);
                    Bitmap bitmap = Bitmap.createBitmap(
                            page.getWidth() * 2, page.getHeight() * 2,
                            Bitmap.Config.ARGB_8888);
                    bitmap.eraseColor(Color.WHITE);
                    page.render(bitmap, null, null,
                            PdfRenderer.Page.RENDER_MODE_FOR_DISPLAY);
                    page.close();
                    renderer.close();
                    showPreviewAndProcess(bitmap);
                }
            } catch (Exception e) {
                showError("PDF error: " + e.getMessage());
            }
        }
    });
  // Camera permission
  private final ActivityResultLauncher<String> permissionLauncher =
        registerForActivityResult(new ActivityResultContracts.RequestPermission(),
granted -> {
            if (granted) launchCamera();
            else Toast.makeText(this, "Camera permission required",
Toast.LENGTH_SHORT).show();
```

```
        });
    @Override
    protected void onCreate(Bundle savedInstanceState)
        { super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_ocr);
        imgPreview       = findViewById(R.id.imgPreview);
        scrollViewOCR    = findViewById(R.id.scrollViewOCR);
        tvResult         = findViewById(R.id.tvResult);
        layoutParameters = findViewById(R.id.layoutParameters);
        Button btnCamera  = findViewById(R.id.btnCamera);
        Button btnGallery = findViewById(R.id.btnGallery);
        Button btnPdf     = findViewById(R.id.btnPdf);
        btnCamera.setOnClickListener(v -> {
            if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
                    == PackageManager.PERMISSION_GRANTED)
                { launchCamera();
            } else {
                permissionLauncher.launch(Manifest.permission.CAMERA);
            }
        });
        btnGallery.setOnClickListener(v -> galleryLauncher.launch("image/*"));
        btnPdf.setOnClickListener(v -> pdfLauncher.launch(new
String[]{"application/pdf"}));
        imgPreview.setOnClickListener(v  ->
            Toast.makeText(this, "Tap Camera/Gallery to scan a new report",
```

```
                Toast.LENGTH_SHORT).show());
}
private void launchCamera()
   { try {
      File photoFile = File.createTempFile(
            "WATER_" + new SimpleDateFormat("yyyyMMdd_HHmmss",
                  Locale.US).format(new Date()),
            ".jpg", getExternalCacheDir());
      cameraImageUri = FileProvider.getUriForFile(
            this, getPackageName() + ".fileprovider", photoFile);
      cameraLauncher.launch(cameraImageUri);
   } catch (Exception e) {
      showError("Cannot open camera: " + e.getMessage());
   }
}
private void showPreviewAndProcess(Bitmap bitmap) {
   if (bitmap == null) { showError("Could not load image."); return; }
   imgPreview.setImageBitmap(bitmap);
   imgPreview.setVisibility(View.VISIBLE);
   processImage(bitmap);
}
private void processImage(Bitmap bitmap)
   { tvResult.setText("Processing via
   cloud...");
   layoutParameters.removeAllViews();
```

```
        try {
            ByteArrayOutputStream baos = new ByteArrayOutputStream();
            bitmap.compress(Bitmap.CompressFormat.JPEG, 80, baos);
            String base64Image = Base64.encodeToString(baos.toByteArray(),
Base64.DEFAULT);
            Map<String, Object> data = new HashMap<>();
            data.put("image", base64Image);
            FirebaseFunctions.getInstance()
                    .getHttpsCallable("extractWaterTestData")
                    .call(data)
                    .addOnSuccessListener(result -> {
                        Map<String, Object> response = (Map<String, Object>) result.getData();
                        if (response == null) {
                            showError("Empty response from server");
                            return;
                        }
                        Map<String, Double> values =
                                (Map<String, Double>) response.get("values");
                        updateUIWithResults(values);
                    })
                    .addOnFailureListener(e -> {
                        showError("Cloud Error: " + e.getMessage());
                    });
        } catch (Exception e) {
```

```
        showError("Processing failed: " + e.getMessage());
    }
}
// RENDER RESULTS
private void updateUIWithResults(Map<String, Double> values)
    { layoutParameters.removeAllViews();
    if (values == null || values.isEmpty()) {
        tvResult.setText("No parameters found. Please try a clearer image.");
        return;
    }
    Map<String, Double> limits = new HashMap<>();
    limits.put("ph",          8.5);
    limits.put("tds",         500.0);
    limits.put("hardness",    200.0);
    limits.put("iron",        1.0);
    limits.put("turbidity",   5.0);
    limits.put("chloride",    250.0);
    limits.put("calcium",     75.0);
    limits.put("magnesium",   30.0);
    limits.put("sulphate",    200.0);
    limits.put("nitrate",     45.0);
    limits.put("fluoride",    1.0);
    limits.put("alkalinity",  200.0);
    limits.put("manganese",   0.1);
```

```
        limits.put("h2s",          0.05);
        for (Map.Entry<String, Double> entry : values.entrySet())
           { View row = getLayoutInflater().inflate(
                 R.layout.item_parameter, layoutParameters, false);

           TextView tvName  = row.findViewById(R.id.tvParamName);
           TextView tvValue  = row.findViewById(R.id.tvParamValue);
           TextView tvStatus = row.findViewById(R.id.tvParamStatus);

           String key   = entry.getKey();
           double value = entry.getValue();

           if (tvName  != null) tvName.setText(key.replace("_", " ").toUpperCase());
           if (tvValue != null) tvValue.setText(value == 0.0 ? "BDL" :
String.valueOf(value));

           if (tvStatus != null)
              { String status;
              int textColor;
              if (value == 0.0)
                 { status  = "BDL";
                 textColor = Color.parseColor("#607D8B");
              } else if (key.equals("ph")) {
                 boolean fail = value < 6.5 || value > 8.5;
                 status    = fail ? "❌ FAIL" : "☑ PASS";
```

```
            textColor = fail ? Color.RED : Color.parseColor("#2E7D32");
        } else {
            Double limit = limits.get(key);
            if (limit == null) {
                status    = "—";
                textColor = Color.parseColor("#607D8B");
            } else if (value > limit)
                { status  = "❌ FAIL";
                textColor = Color.RED;
            } else {
                status    = "☑ PASS";
                textColor = Color.parseColor("#2E7D32");
            }
        }
        tvStatus.setText(status);
        tvStatus.setTextColor(textColor);
    }

    layoutParameters.addView(row);
}

WaterAnalyzer.AnalysisResult analysis = WaterAnalyzer.generateReport(values);
tvResult.setText(analysis.report);
tvResult.setBackgroundColor(analysis.safe
```

```
                ? Color.parseColor("#C8E6C9")
                : Color.parseColor("#FFCDD2"));

        scrollViewOCR.post(() -> scrollViewOCR.fullScroll(View.FOCUS_DOWN));
    }
    private void showError(String msg)
        { tvResult.setText(msg);
        tvResult.setBackgroundColor(Color.parseColor("#FFCDD2"));
    }
}
```

ParserAdavnced.java

```
package com.example.myapplication;
import org.json.JSONArray;
import org.json.JSONObject;
import java.util.HashMap;
import java.util.Map;
public class OCRParserAdvanced {
    public static Map<String, Double> parse(String jsonResponse)
        { Map<String, Double> extractedData = new HashMap<>();
        if (jsonResponse == null || jsonResponse.isEmpty()) return extractedData;

        try {
            // FIX 1: Strip markdown fences AI may accidentally add
```

```
        String clean = jsonResponse
                .replaceAll("(?s)```json\\s*",  "")
                .replaceAll("(?s)```\\s*",      "")
                .trim();
        JSONObject root    = new JSONObject(clean);
        JSONArray  results = root.getJSONArray("results");
        for (int i = 0; i < results.length(); i++) {
            JSONObject obj  = results.getJSONObject(i);
            String rawName  = obj.getString("parameter").toLowerCase().trim();
            double value    = obj.optDouble("value", 0.0); // FIX 2: optDouble won't crash
            String key = normalizeName(rawName);
            extractedData.put(key, value); // FIX 3: always store, never drop a parameter
        }

    } catch (Exception e)
        { e.printStackTrace();
    }
    return extractedData;
}

private static String normalizeName(String name) {
    // FIX 4: Expanded from 7 to 27 parameters covering both report formats
    if (name.contains("ph"))                                return "ph";
    if (name.contains("tds") || name.contains("dissolved solid")) return "tds";
```

```
if (name.contains("hardness"))                    return "hardness";
if (name.contains("iron") || name.contains(" fe"))        return "iron";
if (name.contains("chloride") && !name.contains("residual"))  return "chloride";
if (name.contains("calcium"))                     return "calcium";
if (name.contains("magnesium"))                    return "magnesium";
if (name.contains("turbidity"))                   return "turbidity";
if (name.contains("sulphide") || name.contains("sulfide")
      || name.contains("h2s"))                    return "h2s";
if (name.contains("sulphate") || name.contains("sulfate"))    return "sulphate";
if (name.contains("nitrate"))                     return "nitrate";
if (name.contains("fluoride"))                    return "fluoride";
if (name.contains("alkalinity"))                  return "alkalinity";
if (name.contains("copper"))                      return "copper";
if (name.contains("manganese"))                    return "manganese";
if (name.contains("zinc"))                        return "zinc";
if (name.contains("arsenic"))                     return "arsenic";
if (name.contains("chromium"))                     return "chromium";
if (name.contains("aluminium") || name.contains("aluminum"))  return
"aluminium";
if (name.contains("ammonia"))                      return "ammonia";
if (name.contains("coliform"))                    return "coliforms";
if (name.contains("e.coli") || name.contains("ecoli"))       return "ecoli";
if (name.contains("colour") || name.contains("color"))       return "colour";
if (name.contains("conductivity"))                  return "conductivity";
if (name.contains("boron"))                       return "boron";
```

```
    if (name.contains("residual chlorine"))                    return "residual_chlorine";
    if (name.contains("odour") || name.contains("odor"))       return "odour";
    if (name.contains("taste"))                                return "taste";
    return name.trim().replaceAll("[^a-z0-9]", "_");
  }}
```

waterAnalyzer.java

```
package com.example.myapplication;

import java.util.Map;

public class WaterAnalyzer {

  public static class AnalysisResult
    { public String report;
    public boolean safe;
  }

  public static AnalysisResult generateReport(Map<String, Double> values)
    { StringBuilder sb  = new StringBuilder();
    AnalysisResult result = new AnalysisResult();
    boolean safe = true;

    sb.append("=== WATER QUALITY REPORT ===\n");
```

```
sb.append("Standard: IS 10500:2012\n\n");

// ── pH ──
if (has(values, "ph")) {
    double v = get(values, "ph");
    sb.append("pH              : ").append(v);
    if (v < 6.5 || v > 8.5) { sb.append("  ❌ Unsafe (6.5–8.5)\n"); safe = false; }
    else                    { sb.append("  ☑ Normal\n"); }
} else { sb.append("pH              : Not detected\n"); }

// ── TDS ──
if (has(values, "tds")) {
    double v = get(values, "tds");
    sb.append("TDS             : ").append(v).append(" mg/L");
    if (v > 500) { sb.append("  ❌ High (>500)\n"); safe = false; }
    else         { sb.append("  ☑ Acceptable\n"); }
} else { sb.append("TDS             : Not detected\n"); }

// ── Turbidity ──
if (has(values, "turbidity")) {
    double v = get(values, "turbidity");
    sb.append("Turbidity       : ").append(v).append(" NTU");
    if (v > 5)      { sb.append("  ❌ High (>5 NTU)\n"); safe = false; }
    else if (v > 1) { sb.append("  ⚠ Above 1 NTU desirable limit\n"); }
```

```
    else          { sb.append("  ☑ Clear\n"); }
} else { sb.append("Turbidity        : Not detected\n"); }

// ── Hardness ──
if (has(values, "hardness")) {
    double v = get(values, "hardness");
    sb.append("Hardness        :  ").append(v).append("  mg/L");
    if (v > 200) { sb.append("  ✗ Hard water\n"); safe = false; }
    else      { sb.append("  ☑ Normal\n"); }
} else { sb.append("Hardness          : Not detected\n"); }

// ── Iron ──
if (has(values, "iron")) {
    double v = get(values, "iron");
    sb.append("Iron              : ").append(v).append(" mg/L");
    if (v > 1.0)    { sb.append("  ✗ High (>1.0 mg/L)\n"); safe = false; }
    else if (v > 0.3){ sb.append("  ⚠ Above desirable 0.3 mg/L\n"); }
    else          { sb.append("  ☑ Safe\n"); }
} else { sb.append("Iron              : Not detected\n"); }

// ── Chloride ──
if (has(values, "chloride")) {
    double v = get(values, "chloride");
    sb.append("Chloride          : ").append(v).append(" mg/L");
```

```
    if (v > 250) { sb.append("  ❌ High\n"); safe = false; }
    else         { sb.append("  ☑ Normal\n"); }
} else { sb.append("Chloride          : Not detected\n"); }

// ── Calcium ──
if (has(values, "calcium")) {
    double v = get(values, "calcium");
    sb.append("Calcium           : ").append(v).append(" mg/L");
    if (v > 75) { sb.append("  ⚠ Above limit\n"); }
    else        { sb.append("  ☑ Normal\n"); }
}

// ── Magnesium ──
if (has(values, "magnesium")) {
    double v = get(values, "magnesium");
    sb.append("Magnesium         : ").append(v).append(" mg/L");
    if (v > 30) { sb.append("  ⚠ Above limit\n"); }
    else        { sb.append("  ☑ Normal\n"); }
}

// ── Sulphate ──
if (has(values, "sulphate")) {
    double v = get(values, "sulphate");
    sb.append("Sulphate          : ").append(v).append(" mg/L");
```

```
    if (v > 200) { sb.append("  ❌ High\n"); safe = false; }
    else         { sb.append("  ☑ Normal\n"); }
}

// ── Nitrate ──
if (has(values, "nitrate")) {
    double v = get(values, "nitrate");
    sb.append("Nitrate          : ").append(v).append(" mg/L");
    if (v > 45) { sb.append("  ❌ High\n"); safe = false; }
    else        { sb.append("  ☑ Normal\n"); }
}

// ── Fluoride ──
if (has(values, "fluoride")) {
    double v = get(values, "fluoride");
    sb.append("Fluoride         : ").append(v == 0.0 ? "BDL" : v + " mg/L");
    if (v > 1.0) { sb.append("  ❌ High\n"); safe = false; }
    else         { sb.append("  ☑ Normal\n"); }
}

// ── Alkalinity ──
if (has(values, "alkalinity")) {
    double v = get(values, "alkalinity");
    sb.append("Alkalinity       : ").append(v).append(" mg/L");
```

```
    if (v > 200) { sb.append("  ⚠ Above limit\n"); }
    else         { sb.append("  ☑ Normal\n"); }
}

// ── H2S ──
if (has(values, "h2s")) {
    double v = get(values, "h2s");
    sb.append("H2S / Sulphide    : ").append(v == 0.0 ? "BDL" : v + " mg/L");
    if (v > 0.05) { sb.append("  ✗ Detected (odour risk)\n"); safe = false; }
    else          { sb.append("  ☑ Not detected\n"); }
}

// ── Manganese ──
if (has(values, "manganese")) {
    double v = get(values, "manganese");
    sb.append("Manganese         : ").append(v == 0.0 ? "BDL" : v + " mg/L");
    if (v > 0.1) { sb.append("  ✗ High\n"); safe = false; }
    else         { sb.append("  ☑ Normal\n"); }
}

// ── Coliforms ──
if (has(values, "coliforms")) {
    double v = get(values, "coliforms");
    sb.append("Coliforms         : ");
```

```
    if (v == 0.0) { sb.append("Absent ☑ Safe\n"); }
    else { sb.append(v).append(" MPN/100ml ✗ UNSAFE!\n"); safe = false; }
}

// ── E.Coli ──
if (has(values, "ecoli")) {
    double v = get(values, "ecoli");
    sb.append("E.coli          : ");
    if (v == 0.0) { sb.append("Absent ☑ Safe\n"); }
    else { sb.append(v).append(" MPN/100ml ✗ UNSAFE!\n"); safe = false; }
}

// ── Conductivity ──
if (has(values, "conductivity"))
    { sb.append("Conductivity    : ")
          .append(get(values, "conductivity"))
          .append(" µS/cm\n");
}
sb.append("\n═══════════════════════════════════\n");
if (safe) {
    sb.append("☑ WATER IS SAFE FOR DRINKING\n");
    sb.append("    All tested parameters within IS 10500:2012 limits.");
} else {
    sb.append("✗ WATER MAY NOT BE SAFE\n");
```

```
            sb.append("    One or more parameters exceed IS 10500:2012 limits.\n");
            sb.append("    Recommend treatment before use.");
        }

        result.report = sb.toString();
        result.safe  = safe;
        return result;
    }
    private static boolean has(Map<String, Double> map, String key)
        { return map.containsKey(key) && map.get(key) != null;
    }

    private static double get(Map<String, Double> map, String key)
        { Double val = map.get(key);
        return val != null ? val : 0.0;
    }
}
```